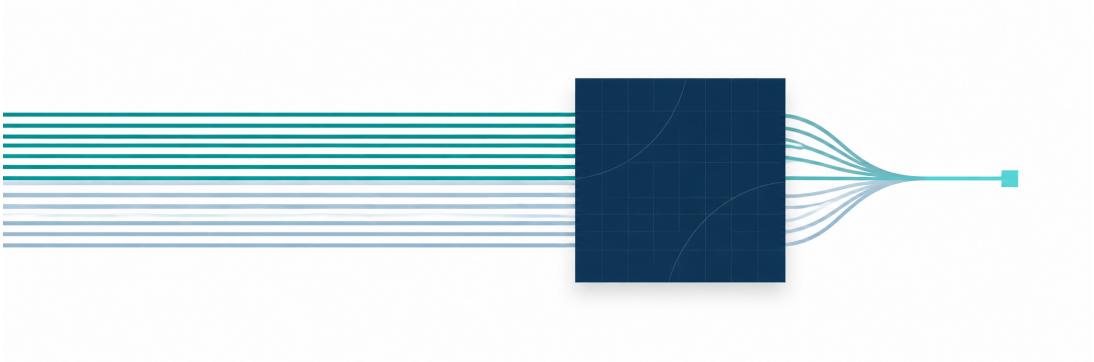
香山昆明湖 V2

自定义向量点积指令

三阶段技术报告合订本

设计、验证与性能评估



三阶段工程技术报告

2026 年 11 月 6 日修订

目录

第一阶段：环境部署与向量执行过程 **1**

1.1 协同验证框架与实验环境 **1**

1.2 部署与构建过程 **3**

1.2.1 资源配置与工具位置 3

1.2.2 固定源码与工具安装 4

1.2.3 集成构建与操作记录 5

1.3 共同启动输入与差分故障定位 **7**

1.4 hello 输出与三类 vadd 结果 **8**

1.4.1 输入配置与结果解释 8

1.4.2 10 月 9 日环境验证 9

1.4.3 10 月 25 日双模型复核 9

1.5 向量执行与波形观察 **10**

1.5.1 10 月 9 日事件与原始截图 10

1.5.2 10 月 25 日含时钟的调试记录 12

1.6 阶段结论与复现索引 **13**

1.6.1 源码与工具版本 13

1.6.2 T04 产物身份 14

1.6.3 文件与记录索引 14

1.7 操作日志摘录 **15**

1.7.1 10 月 9 日工作负载构建 15

1.7.2 10 月 9 日正常运行摘要 15

1.7.3 10 月 25 日生产模型输出 16

1.7.4 10 月 25 日 FST 模型输出 16

第二阶段：指令设计与基础测试 **17**

2.1 设计目标与实施范围 **17**

2.2 指令语义与编码 **17**

2.2.1 八项有符号点积与结果布局 17

2.2.2 数值范围与中间位宽 19

2.2.3 编码与合法执行状态 19

2.3 数据通路与弹性两槽控制 19

2.3.1 乘积槽与结果槽 19

2.3.2 时钟、复位与连续流动 21

2.3.3 背压、重定向与事务守恒 22

2.4 RTL 实现与处理器接入 25

2.4.1 关键实现语句 25

2.4.2 译码、配置依赖与写回 25

2.4.3 共享 VFEX4 的派发与门控 26

2.5 基础功能与分层验证 28

2.5.1 正数点积测试 29

2.5.2 负数点积测试 29

2.5.3 边界值点积测试 30

2.5.4 算术数据通路测试汇总 30

2.5.5 响应配对与复查位置 31

2.6 设计取舍与扩展 32

2.6.1 两槽结构与结果保存 32

2.6.2 十六项 INT8 点积的功能进展 32

2.6.3 元素位宽的参数化验证 33

2.7 实现与验证摘录 33

2.8 源码与证据入口 37

第三阶段：协同仿真与性能评估 38

3.1 从功能验证到工程评价 38

3.2 NEMU 参考语义与处理器接入 39

3.3 差分检验与错误检出 40

3.4 全核波形中的点积事务 43

3.4.1 源数据、结果与身份 43

3.4.2 从有效区间到握手边沿 43

3.4.3 输入与接受 44

3.4.4 结果与响应 45
3.4.5 提交与架构目的 46

3.5 等价 RVV 对照与测量方法 47
3.5.1 完整窗口与配对统计 47

3.6 硬件微基准结果 49

3.7 软件调用与 C8 优化 50

3.8 软件接入与复现说明 52
3.8.1 固定平台与数据布局 52
3.8.2 编译期选择与调用 53
3.8.3 从函数对象到可执行镜像 53

3.9 完整前向计算的组织与验证 55
3.9.1 从点积内核到模型计算链 55
3.9.2 工作量与数据布局 56
3.9.3 完整计算链的验证结果 56

3.10 综合面积、时序与设计成本 57

3.11 设计取舍与应用方向 59

3.12 评估对象与证据索引 60
3.12.1 复现与图像来源 61

第一阶段：环境部署与向量执行过程

本阶段对应赛题的环境部署验证、操作日志和向量加法执行过程。2026 年 10 月 9 日的记录展示环境建立、启动差分修复及三类 vadd 用例；10 月 25 日的 T04 复核进一步完成英文 hello 输出、两种 EMU 模型的差分运行和含时钟的波形调试。下文按两组实验各自的源码、输入和产物说明结果，自定义 vdot 点积的设计与测试见第二阶段。

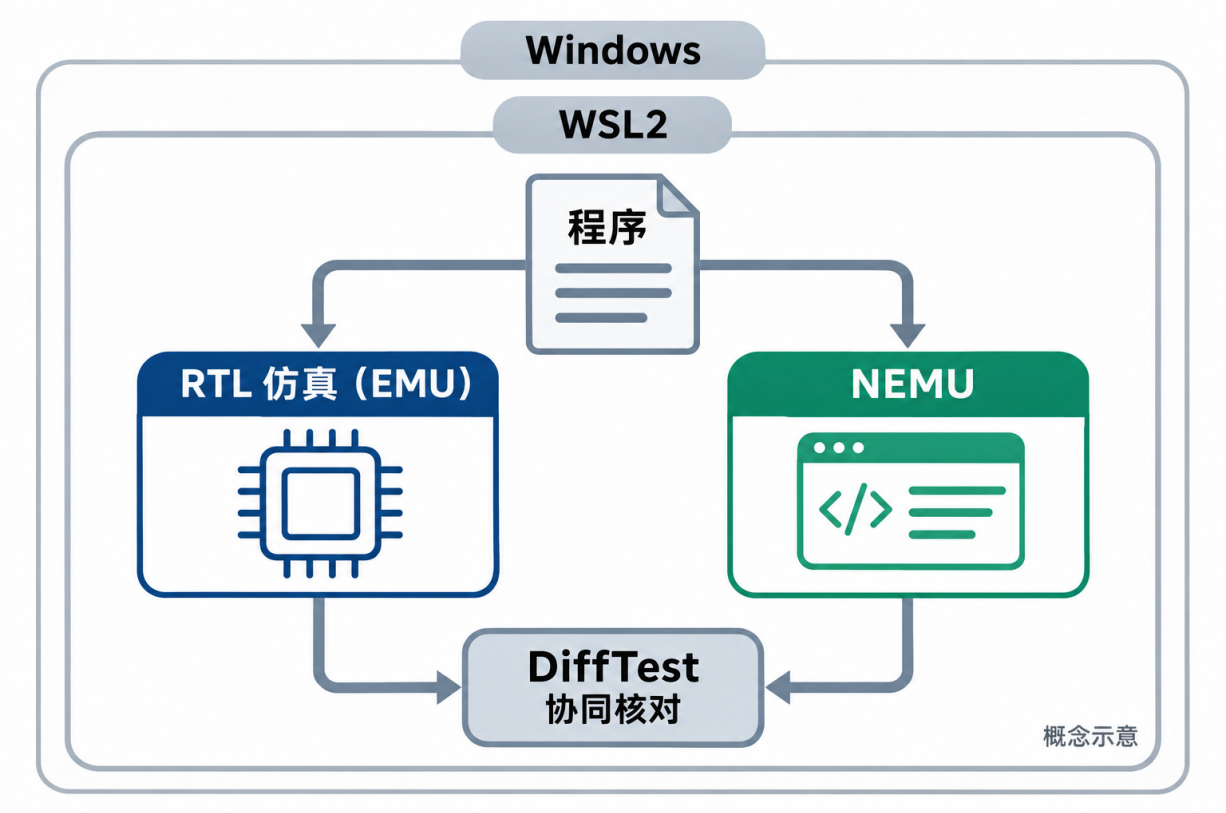
1.1 协同验证框架与实验环境

香山 EMU、NEMU 参考模型和裸机程序共同构成验证环境。EMU 模拟硬件执行，NEMU 按指令语义更新参考状态，DiffTest 在架构状态比较点核对两者。裸机程序提供可重复的输入和结果检查；共同启动 flash 把复位过程连接到程序入口。hello 检查程序启动与串口输出，vadd 则覆盖向量配置、加载、按元素加法、存储和提交。

硬件侧由 Java 与 Mill 解析 Scala/Chisel 工程，firtool 处理硬件中间表示，Verilator 与宿主 C++ 编译器生成 EMU。目标程序使用 RISC-V GCC 和 binutils 编译、链接并生成 ELF、BIN 及反汇编。NEMU 在宿主侧构建为参考库，由运行参数显式加载。各条构建链最终汇合到同一次差分运行，构建记录与运行记录通过产物摘要相连。

表 1-1 构建产物与复核入口

产物	作用	主要记录
香山 EMU	执行 DUT 仿真	硬件源码、DUT 配置、生成命令、编译工具与文件 SHA-256
NEMU 参考库	提供架构状态参考	固定源码、构建配置、实际加载路径与文件 SHA-256
程序 ELF/BIN	执行 hello 和 vadd 用例	ISA/ABI、入口、装载段、反汇编、镜像摘要
共同 flash	建立一致的启动输入	启动汇编、12 字节镜像、反汇编与显式加载参数
运行记录	关联输入、结果及结束状态	原始日志、退出码、波形、事件 CSV 与信号索引



概念示意：共同输入与软硬件状态核对。技术依据见图 1-1。

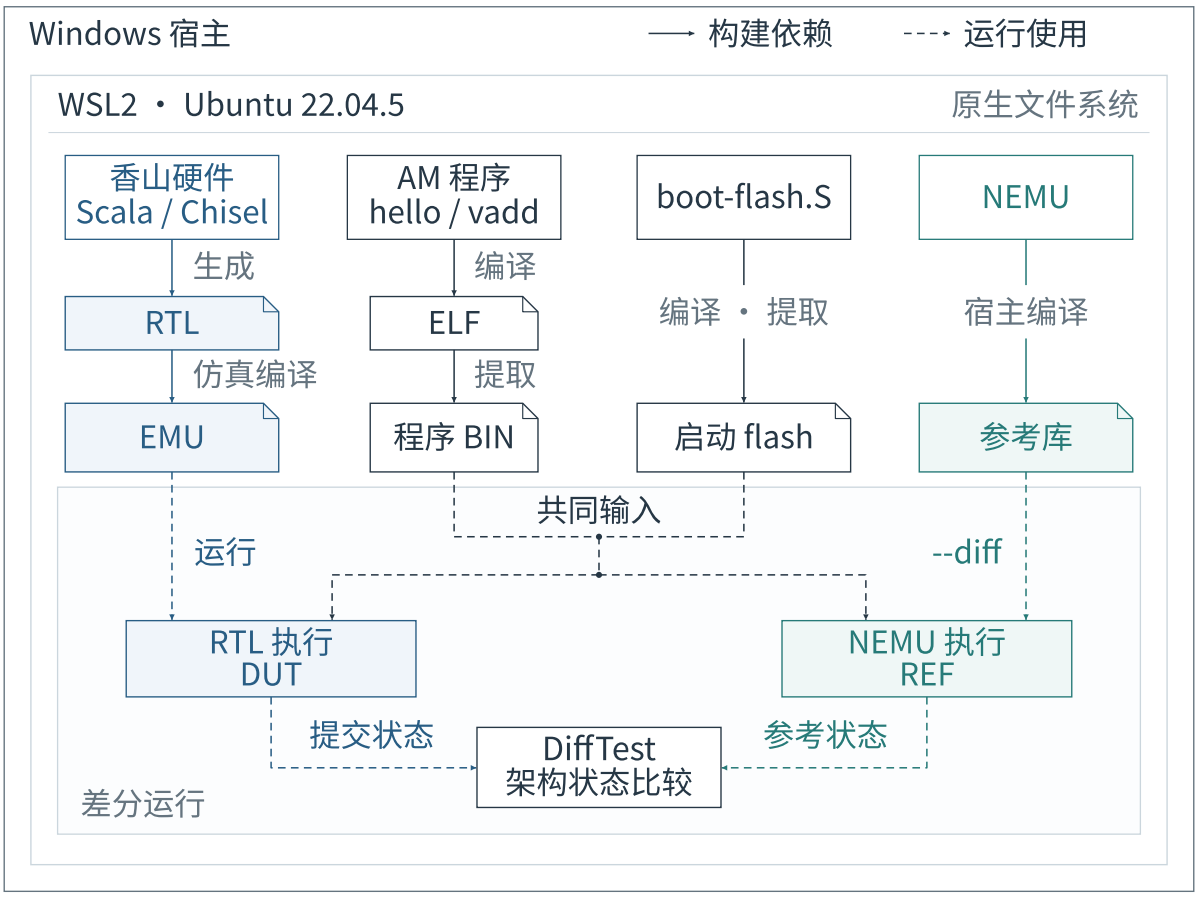


图 1-1 开发环境与软硬件协同验证框架

Windows 宿主中的 WSL2 承载构建与差分运行。实线表示构建依赖，虚线表示运行使用关系；程序 BIN 与启动 flash 作为共同输入，EMU 与 NEMU 分别提供 DUT 和参考状态。

10 月 9 日实验在 Windows 下的 WSL2 Ubuntu 22.04.5 运行，宿主处理器为 Ryzen 7 7840H，可用物理内存约 27.7GiB。WSL 配置 20GB 内存和 16GB swap，构建与仿真放在 WSL 原生文件系统。DUT 为 MinimalConfig、单核、VLEN128，原波形构建采用 EMU_THREADS=1 和 EMU_TRACE=fst。运行时显式指定 128MB RAM、共同 flash 和固定参考库。

本阶段的源码锁、完整工具版本和 T04 产物摘要集中列于本章末尾。10 月 9 日的部署基线与 10 月 25 日复用的 T17 生成物分别标识，便于准确恢复对应实验。

1.2 部署与构建过程

1.2.1 资源配置与工具位置

WSL 资源设置位于 Windows 用户目录的%UserProfile%\.wslconfig：

```
[wsl2]
memory=20GB
swap=16GB
```

保存正在进行的 WSL 工作后，在 PowerShell 执行`wsl --shutdown`，重新进入 Ubuntu 并用`free -h`核对。原机器的交换文件位于 D:\WSL\wsl-swap.vhdx，其他机器按本机磁盘布局配置。内

存规划同时覆盖 JVM、硬件生成工具、C++ 编译进程和文件缓存；频繁换页会增加构建时间，因此应根据可用物理内存调整 JVM 堆与编译并发。

Windows 工作树的 Git 检查、提交与推送使用 Windows 原生 Git；Linux/WSL 原生目录使用 Linux Git。Scala/Chisel、RISC-V 程序、EMU/NEMU 和波形处理在 Linux 原生目录执行。源码与产物跨环境交换时，在接收端核对完整提交号、未提交差异、外部依赖及关键文件 SHA-256，运行输出使用独立目录。

这一安排也对应实际工程经验：2026 年 10 月 24 日 R4 发布检查中，一次从 WSL 访问 Windows 工作树的检查约 300 秒后超时；核实未提交且没有残留进程或索引锁后，Windows 原生 Git 完成了后续检查与提交。具体过程见协调记录，该记录用于说明执行位置的选择。

1.2.2 固定源码与工具安装

在团队仓库执行`bash team/scripts/prepare-dependencies.sh`，按版本锁初始化 xs-env、NEMU、nexus-am、DRAMsim3 及递归依赖。团队仓库的香山子模块使用`git clone --recurse-submodules`恢复；NEMU 的 SimPoint 按子模块记录初始化，其余下载依赖按锁定脚本检出。已有目录的版本或已跟踪修改不一致时，先保留现场并核对输入。构建前记录实际仓库根、分支、完整 HEAD、Git common-dir、修改状态与正在使用的输入输出，避免与其他任务共用可写 Git 索引或覆盖冻结产物。

基础工具的安装入口如下。这里的命令用于恢复固定环境，安装后再核对实际工具版本。

```
source team/scripts/activate.sh
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target base
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target llvm
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target mill
sudo apt-get install -y cmake ninja-build gtkwave
```

该入口来自固定 xs-env 的setup-tools.sh。软件源中的补丁版本会变化，环境检查负责记录安装后实际使用的版本。Mill 读取仓库的.mill-version 选择项目版本，Scala、Chisel 和 firtool 按项目解析结果核对。

原 JDK 安装入口使用 Oracle 21/latest 地址。团队的固定安装脚本改用 21.0.12 Linux x64 归档及官方 SHA-256，安装到用户目录：

```
export JAVA_HOME="$HOME/.local/share/graalvm-jdk-21.0.12"
bash team/scripts/install-graalvm.sh
source team/scripts/activate.sh
```

新终端沿用相同 JAVA_HOME。原/opt/graalvm-jdk-21 安装可在版本核对后继续使用；脚本保留版本不同的已有安装。参考资料为Oracle 安装说明、固定归档和官方校验值。

Verilator 从固定提交构建，显式选择已安装的 Clang，并按资源情况控制编译并发：

```
sudo apt-get install -y help2man perl autoconf flex bison
libgoogle-perftools-dev \
  libjemalloc-dev numactl perl-doc zlib1g-dev liblz4-dev libfl2 libfl-dev
mkdir -p "$HOME/riscv-tools"
git clone https://github.com/verilator/verilator.git
"$HOME/riscv-tools/verilator"
cd "$HOME/riscv-tools/verilator"
git checkout ea338be98e1e838d3518809ce8899f85a009963c
unset VERILATOR_ROOT
autoconf
```

```
./configure CC=/usr/local/llvm/bin/clang CXX=/usr/local/llvm/bin/clang++
LINK=/usr/local/llvm/bin/clang++
make -j2
sudo make install
```

安装后回到团队仓库根目录，运行 activate.sh 和 check-environment.sh。原环境目录为/home/riscv/riscv-lab/xs-env/XiangShan；activate.sh 将实际团队仓库设为 NOOP_HOME，依据 TEAM_XS_ENV_ROOT 选择 NEMU 和 AM，并设置 LINUX_GNU_TOOLCHAIN=1。向量工作负载构建显式使用`MARCH=rv64gcv`。

1.2.3 集成构建与操作记录

确认源码、工具和输出目录后，依次构建参考库、EMU、目标程序及共同 flash，再执行差分运行：

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
bash team/scripts/stage1.sh nemu
bash team/scripts/stage1.sh emu
TEAM_ID= 正式团队标识 bash team/scripts/stage1.sh workload
bash team/scripts/stage1.sh run
```

概念示意：部署、运行与证据归档。技术依据见图 1-2。

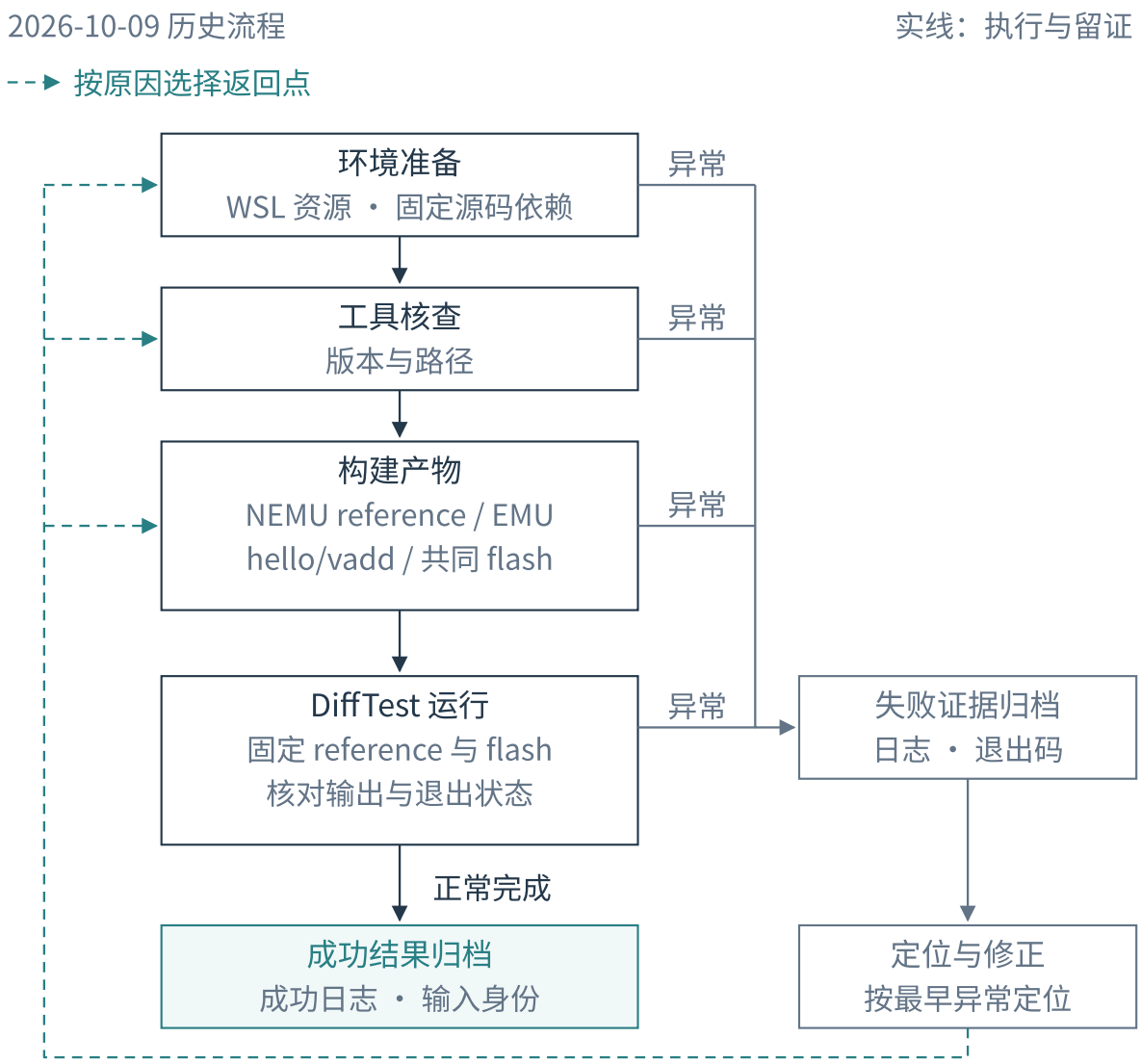


图 1-2　环境部署与验证执行流程

环境准备、工具核查、构建、DiffTest 运行和证据归档构成主流程。失败记录单独保存，定位后按原因返回相应环节。

10 月 9 日构建按实际资源调整并行度，过程保留在原始日志中。下表时间沿用日志的 UTC+8 时区。

表 1-2　2026 年 10 月 9 日主要操作记录

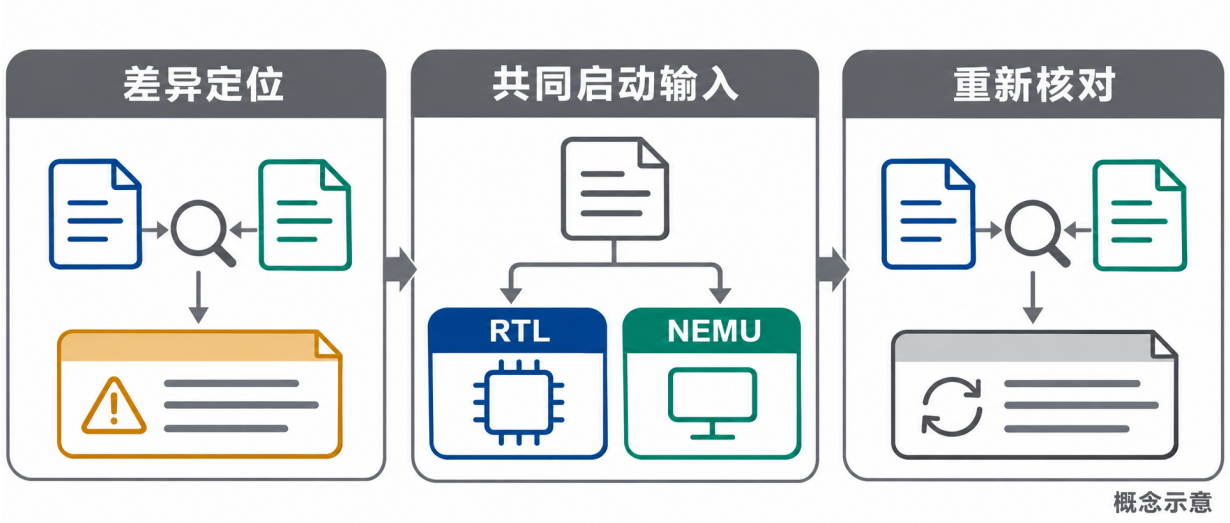
时间	操作与结果
20:42:15	中断并恢复 C++ 构建，并行度调整为 4，保留已完成对象。
20:53:32	EMU 构建完成，状态文件记录 exit_code=0。
20:55:26	工作负载与共同 flash 构建完成，记录 WORKLOAD_BUILD_PASS。
21:00:28	正常差分运行结束，退出状态为 0。

编译并发用于分配宿主资源，DUT 核数和 EMU 仿真线程数由模型配置决定。原 C++ 构建的`-j4`与工具安装示例的`-j2`对应不同步骤。复用对象文件时，源码、工具与生成参数随构建记录一起保存；长作业另记录资源峰值、日志大小、墙钟限额和进程结束状态。

1.3 共同启动输入与差分故障定位

初次运行在 PC0x10000004 出现 t0 寄存器差异，记录的 instrCnt 为 2，尚处于启动代码。比较默认 flash 和反汇编后发现，两端分别使用了 3 条与 6 条启动指令。修复由同一份 boot-flash.S 生成 12 字节 boot-flash.bin，并通过运行参数显式提供给 DUT 与 NEMU。随后两端以共同启动输入进入同一工作负载，在 DiffTest 启用的条件下完成 hello/vadd 检查。

ELF 保存符号、装载段和调试资料，BIN 提供程序字节，flash 提供复位后的启动代码。定位波形时，用与被测 BIN 同次构建的反汇编核对 PC 和机器字；链接参数或库布局改变后，应重新建立这组对应关系。flash 中的文件偏移与处理器执行地址通过各端装载映射关联。



概念示意：统一启动输入与差分恢复。技术依据见图 1-3。

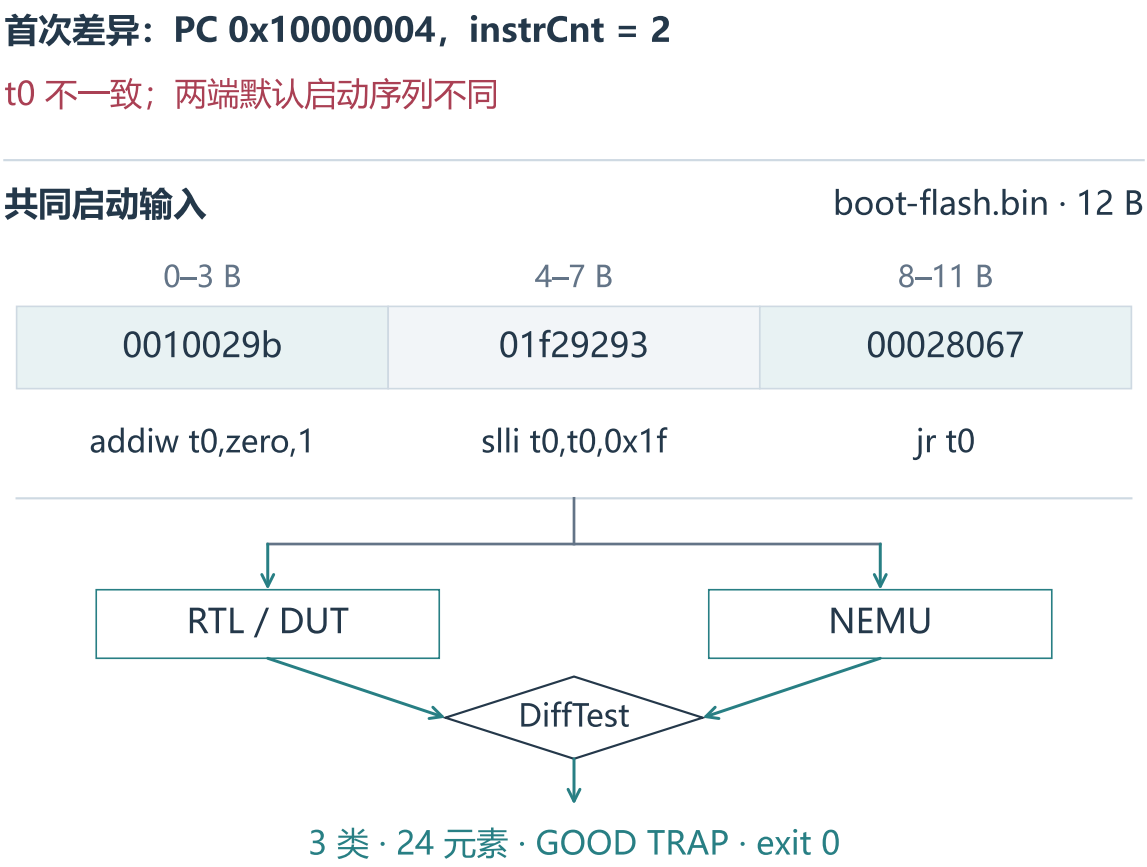


图 1-3　共同启动镜像与差分恢复

首次差异位于 PC 0x10000004、instrCnt=2。共同启动文件由三条机器指令组成，图中 0–3、4–7 和 8–11 标示文件字节偏移。显式加载同一 12 字节 flash 后，三类共 24 个元素完成比较，并以 GOOD TRAP、exit 0 结束。

1.4　hello 输出与三类 vadd 结果

1.4.1　输入配置与结果解释

vadd 程序设置 SEW8、LMUL1、VL8，执行两次`vle8.v`、一次`vadd.vv`和一次向量存储，再逐字节比较标量参考。三个用例检查基本加法、负数补码和 8 位回绕，合计比较 24 个活动元素。

表 1-3　vadd 输出的两种表示

用例	按元素顺序输出的字节	结果低 64 位
正数相加	9 9 9 9 9 9 9 9	0x0909090909090909
负数补码	254 252 250 248 246 244 242 240	0xf0f2f4f6f8fafcfe
边界回绕	128 127 0 0 0 0 254 254	0xfefe000000007f80

每个元素按模 256 保留低 8 位。例如打印值 254 对应十六进制 FE，按 8 位补码解释为 −2。波形将多个字节合并显示为十六进制，右端为最低字节；日志按元素序号递增输出，因此两种书写顺序应按字节位置对应。VLEN128 给出寄存器宽度，VL8 给出本用例的活动元素数，表中结果覆盖低 64 位的八个元素。

1.4.2 10 月 9 日环境验证

10 月 9 日 hello 输出使用 riscv-vdot-team，身份状态为 placeholder，显示的 IPv4 为 10.198.93.97，对应当时 WSL 宿主。正常记录包含 HELLO_VADD_PASScases=3lanes=24、DiffTest 启用标记、GOOD TRAP 和退出状态 0；整程序统计为 16557 条指令、42962 周期，覆盖启动、UART 和结果校验。

Historical hello/vadd output; original run 2026-10-09

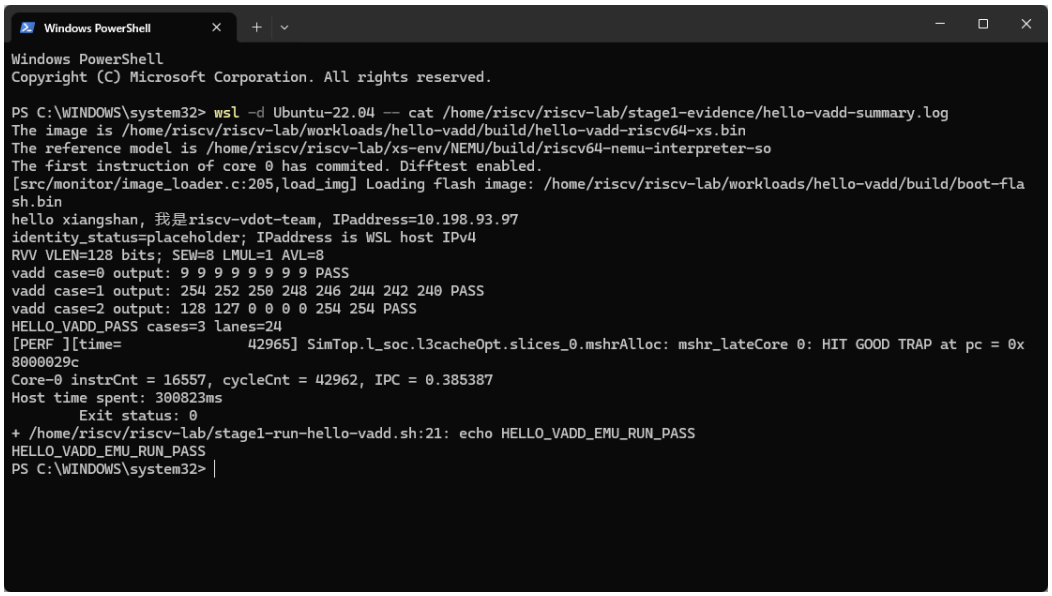


图 1-4 hello 与 vadd 正常结束输出

日志展示三类 vadd 结果、DiffTest 运行和 GOOD TRAP 终态，输出中的身份与地址按原实验保留。

1.4.3 10 月 25 日双模型复核

10 月 25 日（UTC+8）的 T04 实验以个人标识 LiZufan001 构建英文 hello 程序，输出明确标注地址来自构建宿主：

```
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
```

同一 BIN 分别在封存生产 EMU 和独立 FST 模型上运行，均加载普通 NEMU 参考库并启用 DiffTest。两次运行都通过三组、24 个元素检查，命中 GOOD TRAP 并正常退出；每次为 16621 条指令、43249 周期。这里的周期同样对应完整 hello/vadd 程序。表内实际起止时间来自各自运行回执。

表 1-4 T04 双模型运行记录（2026-10-25，UTC+8）

模型	实际起止时间	运行结果
生产 EMU	01:09:21.574716–01:10:18.357888	DiffTest 通过，GOOD TRAP，exit 0
独立 FST 模型	01:28:57.974329–01:31:24.009122	DiffTest 通过，GOOD TRAP，exit 0

新 FST 模型复用 T17 的 1807 个逐文件核验输入；最终核对原件及副本均无变化。发布源码重新编译得到的 BIN 和 flash 与实际执行输入逐字节相同，原始 ELF 和发布 ELF 分别留存。对应验收记录、生产运行回执和FST 运行回执均随证据保存。

1.5 向量执行与波形观察

向量指令经取指、译码、重命名与调度进入执行单元。两个`vle8.v`提供输入，`vadd.vv`产生结果，随后经过写回与 ROB 提交。本文用提交 PC0x80000138、机器字 0x021101d7 及`vecwen=1`，`wdest=3`关联三类用例；执行结果出现和架构提交分别作为观察点记录。

1.5.1 10 月 9 日事件与原始截图

以下数值沿用原 FST 提取 CSV 的事件坐标。局部截图未显示时钟，显示单位为 ps；本表用于定位输入、输出和提交，指令时延与性能通过专门测量窗口评估。

表 1-5 10 月 9 日 vadd 事件坐标

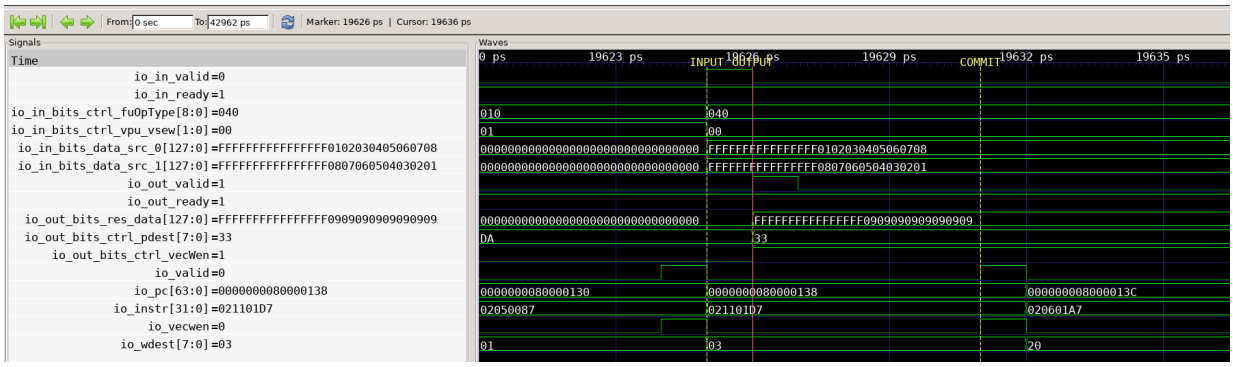
用例	输入	输出	提交	输出低 64 位
正数	19625	19626	19631	0x0909090909090909
负数	25460	25461	25466	0xf0f2f4f6f8fafcfe
边界	33009	33010	33015	0xfefe000000007f80

概念示意：向量操作的执行与状态观察。技术依据见图 1-5。

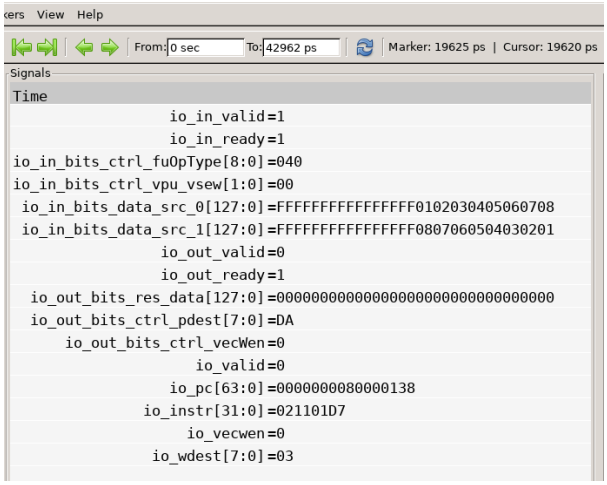
图 1-5 vadd 输入、输出与提交事件时间线

三类用例按指令身份关联执行端输入、结果输出和 ROB 提交；数值沿用事件 CSV 坐标。

(a) Output handshake, marker 19626 ps; all three event positions visible



(b) Input marker 19625 ps



(c) Commit marker 19631 ps

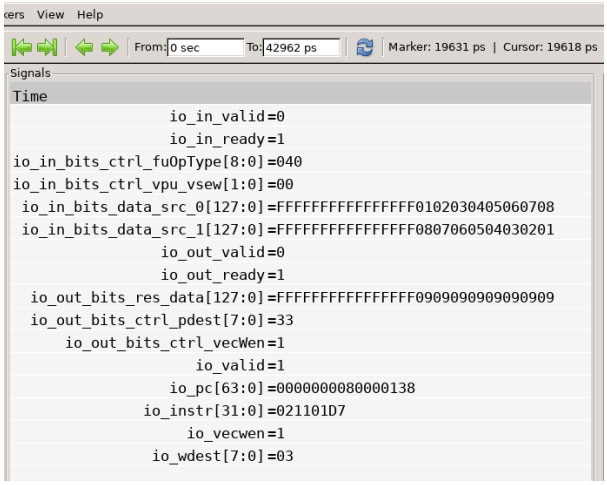
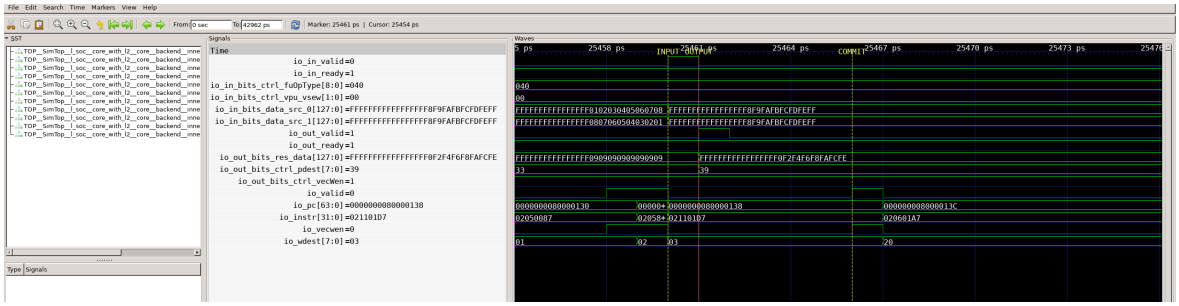


图 1-6 vadd 正数用例的执行与提交波形

(a) 输出事件及相关位置；(b) 输入游标处信号值；(c) 提交游标处信号值。19625、19626 和 19631 对应表中的三个事件坐标。

(a) Negative case: output marker 25461 ps



(b) Boundary case: output marker 33010 ps

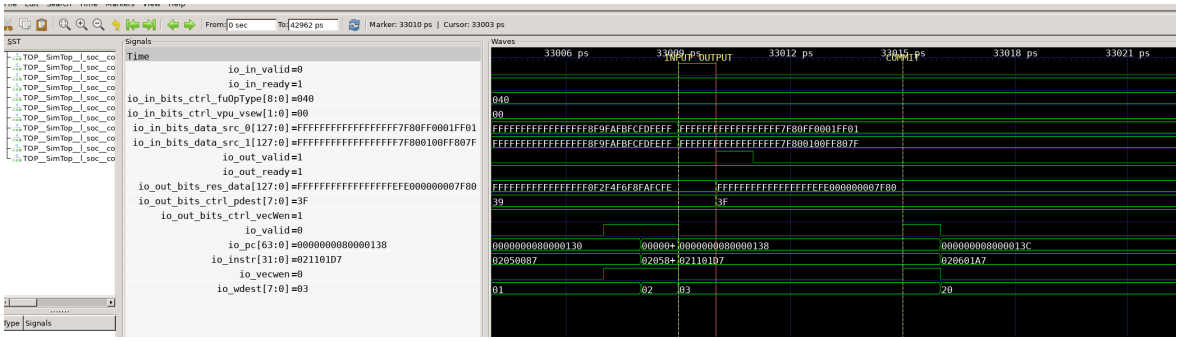


图 1-7 vadd 负数与边界用例结果波形

负数和边界样例输出低 64 位分别为 F0F2F4F6F8FAFCFE 与 FEFE000000007F80，游标对应各自输入、输出和提交位置。

原完整 FST 路径记录为/home/riscv/riscv-lab/stage1-evidence/hello-vadd.fst，大小 220295598 字节。复核可先从提交 PC 定位同次反汇编，再根据 CSV 打开波形窗口，最后联合查看时钟、valid/ready、操作数、结果及事务标识。信号层级与位宽由 evidence/wave-signals.json 提供。

1.5.2 10 月 25 日含时钟的调试记录

T04 的独立 FST 模型保留实际时钟信号，最终选择两个 VialuFix、八个提交槽和 127 个信号，检查 33249 个完整上升沿，得到三次 FU 输入、三次 FU 输出和三次 DPIC 提交观测，共九条事件。三个运算均在 exus_2.VialuFix 发生，同时检查了 exus.VialuFix。

采样取每个局部上升沿前的稳定低相值，周期字段取当时的 DifftestTrapEvent.io_cycleCnt，FST 原始时戳另存。DPIC 记录经过 Rob.scala 中`delay=3` 的观察路径，表内 DPIC 列因此表示提交接口的观测时刻。

表 1-6 T04 按时钟沿采样的事件记录

用例	FU 输入 timer	FU 输出 timer	DPIC 观测 timer	ROB flag/value/pdest
正数	19525	19526	19531	1 / 13 / 51
负数	25526	25527	25532	0 / 21 / 57
边界	33193	33194	33199	1 / 25 / 63

三类数值结果与表1-3一致。完整采样定义、信号清单、原始时戳及摘要保存在wave.json；选通信号 FST 可直接用 GTKWave 查看。接口与观察路径分别对应固定源码的VIAluFix 包装

层和Rob.scala。

工具检查共 24 项，覆盖日志完整性、实际退出状态、输入身份、关键结果，以及缺 FU、缺半周期、未知有效操作数和错误事务标识等反例。首次 FST 构建的工具路径问题和首版提取器遗漏无数字后缀 FU 的记录均保留；修正提取范围后，从同一 FST 完成最终九事件核对。这些检查验证记录与提取工具，实际硬件输入仍为上述三组 vadd 用例。

1.6 阶段结论与复现索引

阶段一完成了环境部署、共同启动输入配置、三类向量加法的差分验证和可定位的波形观察链。10 月 9 日记录说明环境建立过程，10 月 25 日记录补充配置身份的英文 hello 输出与含时钟的执行观察。后续阶段在这一环境上开展 vdot 语义、RTL 和应用性能验证。

1.6.1 源码与工具版本

下表为 10 月 9 日部署时固定的源码身份，完整递归依赖见 evidence/submodules-after-build.lock.txt。

表 1-7 10 月 9 日源码版本锁

组件	提交号
xs-env	b0b506481e839a5df8a580e76f5aa246dc60fe7c
香山基线	16ae9ddcda54fc9a2fddffad73174cf793ac7814
NEMU	a48a11f208ff81ad765591d83271d1b339fb3861
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
DRAMsim3	1d2a9bf6da8e050d975ef2669c42faa93943f489
DiffTest	62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0
yunsuan	cadd3c2f43096e253d61296b33ac697be8354e29

表 1-8 10 月 9 日工具版本

工具	版本
Oracle GraalVM JDK / javac	21.0.12
LLVM / Clang	19.1.7
Mill	项目版本 0.12.3
Scala / Chisel	2.13.15 / 6.6.0
firtool	1.62.1
Verilator	5.052，提交 ea338be98e1e838d3518809ce8899f85a009963c
RISC-V GCC / GNU binutils	11.4.0（riscv64-linux-gnu-前缀）/ 2.38
宿主 GCC / G++	11.4.0
Make / CMake / Ninja	4.3 / 3.22.1 / 1.10.1
Python / GTKWave	3.10.12 / 3.3.104

完整版本输出见 evidence/tool-versions.txt；Ubuntu 软件包清单保留原环境的软件来源。新环境按所需工具安装并核对版本，硬件改动的验证结果再绑定各自构建产物。

1.6.2 T04 产物身份

10 月 25 日 T04 运行使用的 T17 硬件生成源提交为 34bae6d4acceac939f958199dae53c14bc1f6183。以下摘要直接对应 evidence/stage1-current-20261025/中的运行和验收记录。

表 1-9 T04 主要产物 SHA-256

对象	SHA-256
生产 EMU	ff840528eb1009e94906eefb3773ca321050f309553543ab26f0db23c410e30f
FST 模型	1ff3f457ff582d0c3155a15dafa1fa60b8275d9356e29a0b7806c9d3915ce9f7
普通 NEMU 参考库	8af118e5519db7ef6001010a2d547707c5d47630412e80abfc3f343bfef987c9
共同程序 BIN	d81437b44473cef2b0c5777330e13fd076567d4fc1a4296be35d971e51a4dcad
共同 flash	a3360212cf7bb784119c1719c9258362f8567803601b2359aee2c104864fa994
完整 FST（181143183 字节）	cf8abe9104fb955c7dc4bf1efda6588a61060fce33857b9ddc6d5020421be299
选通信号 FST（393288 字节）	4c9f442704871010026654fd9d60854024b270a83be6354be5dee3ff25df24e1

1.6.3 文件与记录索引

运行源码位于 team/workloads/hello-vadd。10 月 9 日证据归档位于 team/evidence/stage1-20261009，T04 证据位于 team/evidence/stage1-current-20261025；报告源码的 evidence/目录保存用于本文的对应日志与索引。完整 FST 按记录中的路径和摘要定位。

表 1-10 报告附件索引

报告目录中的记录	用途
evidence/workload-build.log	工作负载、flash 构建命令与成功状态。
evidence/hello-vadd-summary.log	10 月 9 日身份、三组结果、DiffTest 和退出摘要。
evidence/rendered/nemu-build.txt	NEMU 构建日志的可读副本。
evidence/emu-build-20261009-204228.log	EMU 构建与并行恢复过程。
evidence/tool-versions.txt	工具版本原始输出。
evidence/submodules-after-build.lock.txt	递归依赖完整提交锁。
evidence/vadd-commit-events.csv	配置、加载、vadd 及后续指令的提交记录。
evidence/vector-fix-alu-events.csv	三组向量 ALU 输入与输出记录。
evidence/wave-signals.json	10 月 9 日信号层级、位宽与标识。
evidence/stage1-current-20261025/production-run/run.json 与 trace-run/run.json	T04 两次运行的命令、输入前后摘要、退出状态和日志摘要。
evidence/stage1-current-20261025/wave/wave.json	T04 采样定义、127 个信号、九条事件及波形摘要。

报告目录中的记录	用途
evidence/stage1-current-20261025/acceptance.json 与 RESULT.md	T04 输入等价、24 项工具检查与完整复现索引。

复核时先检查运行回执和通过摘要，再核对程序、reference 与 flash 身份，最后用反汇编和事件 CSV 定位波形。详细操作入口见阶段一记录、环境部署方法和T04 复现说明。

1.7 操作日志摘录

本节保留主要运行摘要与构建完成片段；完整构建日志、版本锁和事件 CSV 随源码包提供，路径见表1-10。

1.7.1 10 月 9 日工作负载构建

```
+ /home/riscv/riscv-lab/stage1-build-workload.sh:20: make ARCH=riscv64-xs
MARCH=rv64gcv -j1
# Building hello-vadd [riscv64-xs] with AM_HOME
{/home/riscv/riscv-lab/xs-env/nexus-am}
+ CC main.c
# Building lib-am [riscv64-xs]
+ AS src/nemu/common/mainargs.S
+ AR -> build/am-riscv64-xs.a
# Building lib-klib [riscv64-xs]
# Creating binary image [riscv64-xs]
+ LD -> build/hello-vadd-riscv64-xs.elf
+ OBJCOPY -> build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:21: riscv64-linux-gnu-gcc -c
-march=rv64gc -mabi=lp64d boot-flash.S -o build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:22: riscv64-linux-gnu-objcopy
-O binary -j .text build/boot-flash.o build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:23: riscv64-linux-gnu-objdump
-d build/boot-flash.o


+ /home/riscv/riscv-lab/stage1-build-workload.sh:28: echo WORKLOAD_BUILD_PASS
WORKLOAD_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-workload.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: printf 'exit_code=%s
end=%s\n' 0 2026-10-09T20:55:26+08:00
```

1.7.2 10 月 9 日正常运行摘要

```
The image is
/home/riscv/riscv-lab/workloads/hello-vadd/build/hello-vadd-riscv64-xs.bin
The reference model is
/home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/riscv/riscv-lab/workloads/hello-vadd/build/boot-flash.bin
hello xiangshan, 我是 riscv-vdot-team, IPaddress=10.198.93.97
identity_status=placeholder; IPaddress is WSL host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
```

```
HELLO_VADD_PASS cases=3 lanes=24
[PERF ][time=               42965] SimTop.l_soc.l3cacheOpt.slices_0.mshrAlloc:
mshr_lateCore 0: HIT GOOD TRAP at pc = 0x8000029c
Core-0 instrCnt = 16557, cycleCnt = 42962, IPC = 0.385387
Host time spent: 300823ms
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-run-hello-vadd.sh:21: echo
HELLO_VADD_EMU_RUN_PASS
HELLO_VADD_EMU_RUN_PASS
```

1.7.3 10 月 25 日生产模型输出

```
emu compiled at Oct 24 2026, 09:22:45
Using simulated 32768B flash
use /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/」
boot-flash.bin as flash bin
Using simulated 128MB RAM
The image is /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/worklo」
ad/build/hello-vadd-riscv64-xs.bin
The reference model is /home/lizufan/riscv-lab/t17-clean-20261024-0854/support」
-01/nemu/reference/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image: /home/lizufan/r」
iscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin

[src/monitor/image_loader.c:250,load_img] Read 12 bytes from file /home/lizufa」
n/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin to
0x0xf0000000
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu
loads a snapshot)
Host time spent: 56742ms
```

1.7.4 10 月 25 日 FST 模型输出

```
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu
loads a snapshot)
Host time spent: 145975ms
```

第二阶段：指令设计与基础测试

2.1 设计目标与实施范围

点积把两组数据逐项相乘，再将乘积求和，是量化矩阵运算的重要计算步骤。本设计在昆明湖 V2 已有向量路径中增加八项 INT8 点积指令，将乘法与归约组合为专用数据通路，同时复用处理器的译码、寄存器重命名、调度、写回和精确提交机制。

通用构建默认采用单请求实现，弹性两槽实现作为可选优化保留。本章重点介绍两槽实现的算术结构、背压和取消处理，并通过正数、负数及边界值测试展示结果。第三阶段的主要周期和成本数据对应明确标识的八项两槽受测版本；十六项扩展的最新功能验证列在本章末尾。这样的组织覆盖赛题 2.1 的设计说明、2.2 的 RTL 实现和 2.3 的三类基础测试。

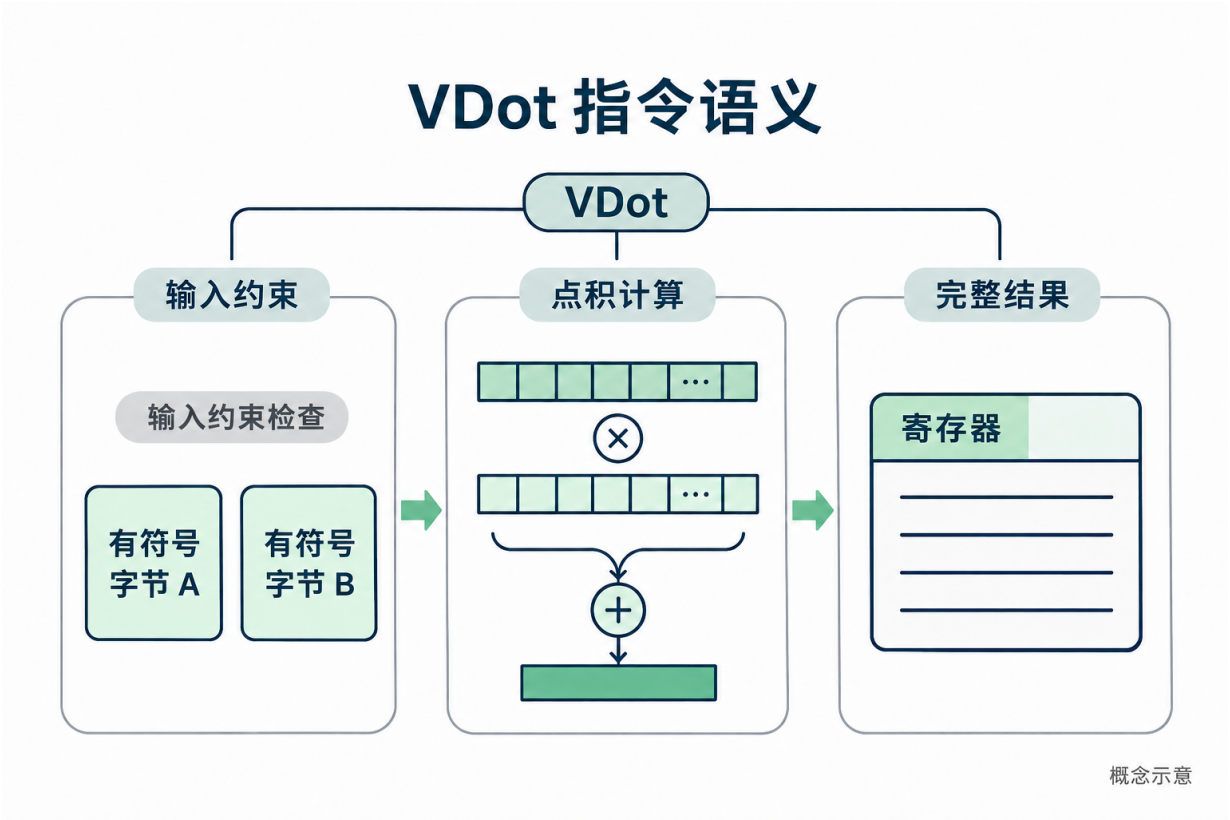
2.2 指令语义与编码

2.2.1 八项有符号点积与结果布局

本指令采用 128 位向量寄存器（VLEN=128）、8 位元素（SEW=8）和单寄存器分组（LMUL=1）；有效元素数为 8（VL=8），起始执行索引为 0（vstart=0）。指令读取两个源寄存器的低 8 个字节，将每个字节解释为有符号整数，计算

$$S = \sum_{i=0}^{7} \mathrm{int8}(vs1[8i+7:8i])\,\mathrm{int8}(vs2[8i+7:8i]).$$

目的低 32 位保存 S 的二补码，高 96 位写零。运算从零开始累加，读取指令执行前的源值，因此支持目的与源寄存器重合，也支持使用 v0 作为普通源或目的。每个源以 128 位向量传入数据通路，八项算术使用其中低 64 位的 8 个元素。



概念示意：输入约束、点积与结果写回。技术依据见图 2-1。

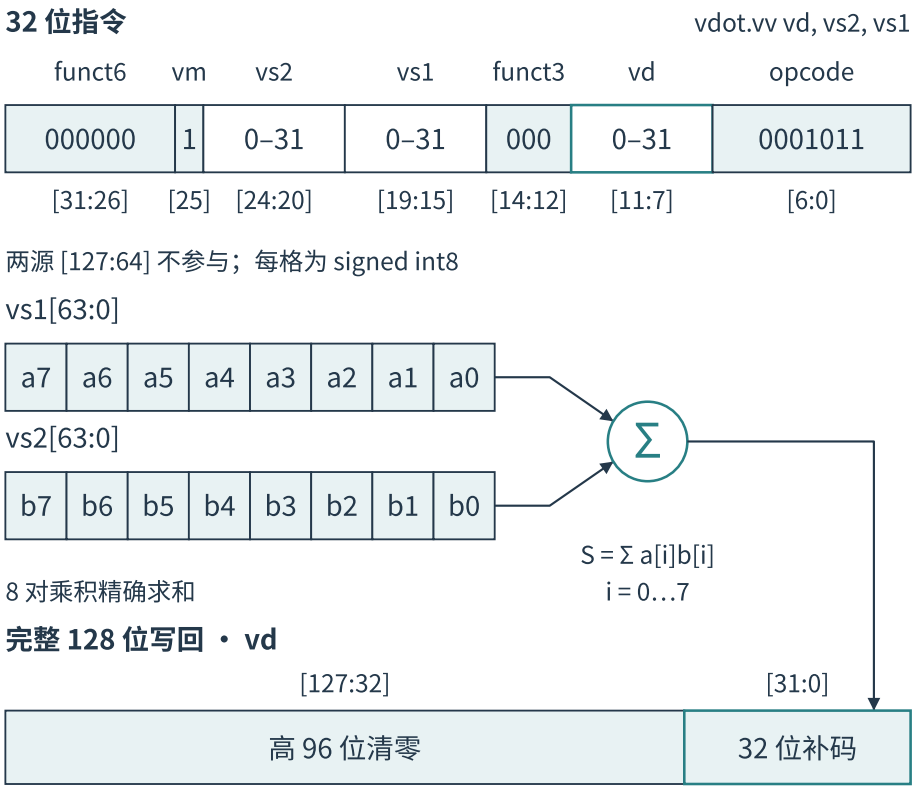


图 2-1　八项点积指令字段与完整结果布局

两个源各提供 8 个有符号字节；目的低 32 位保存点积结果，高 96 位写零。

2.2.2 数值范围与中间位宽

有符号 8 位数的范围为 -128 至 127，单个乘积范围为 $[-16256, 16384]$，八项和范围为 $[-130048, 131072]$。乘积使用 16 位；按二项、四项、八项归并时，精确位宽依次增加为 17、18、19 位。正端点 131072 比 18 位有符号数的最大值大 1，因此极值用例能直接检验归约过程中是否保留了完整进位。

归约结果先符号扩展为 32 位，再与 96 位零拼接。负数的符号由低 32 位字段表达，例如 -8 的低 32 位为 `0xfffffff8`，完整 128 位结果仍以 96 个零作为高位。内部精确和的 19 位、结果槽的 32 位和公开写回的 128 位分别承担算术、存储和架构接口三种作用。

2.2.3 编码与合法执行状态

指令采用 custom-0 编码，`MATCH=0x0200000b`、`MASK=0xfe00707f`。

字段	位段	含义或取值
funct6	31:26	`000000`
vm	25	`1`
vs2、vs1	24:20、19:15	两个向量源寄存器编号
funct3	14:12	`000`
vd	11:7	向量目的寄存器编号
opcode	6:0	custom-0，`0x0b`

汇编记法为 `vdot.vv vd, vs2, vs1`。

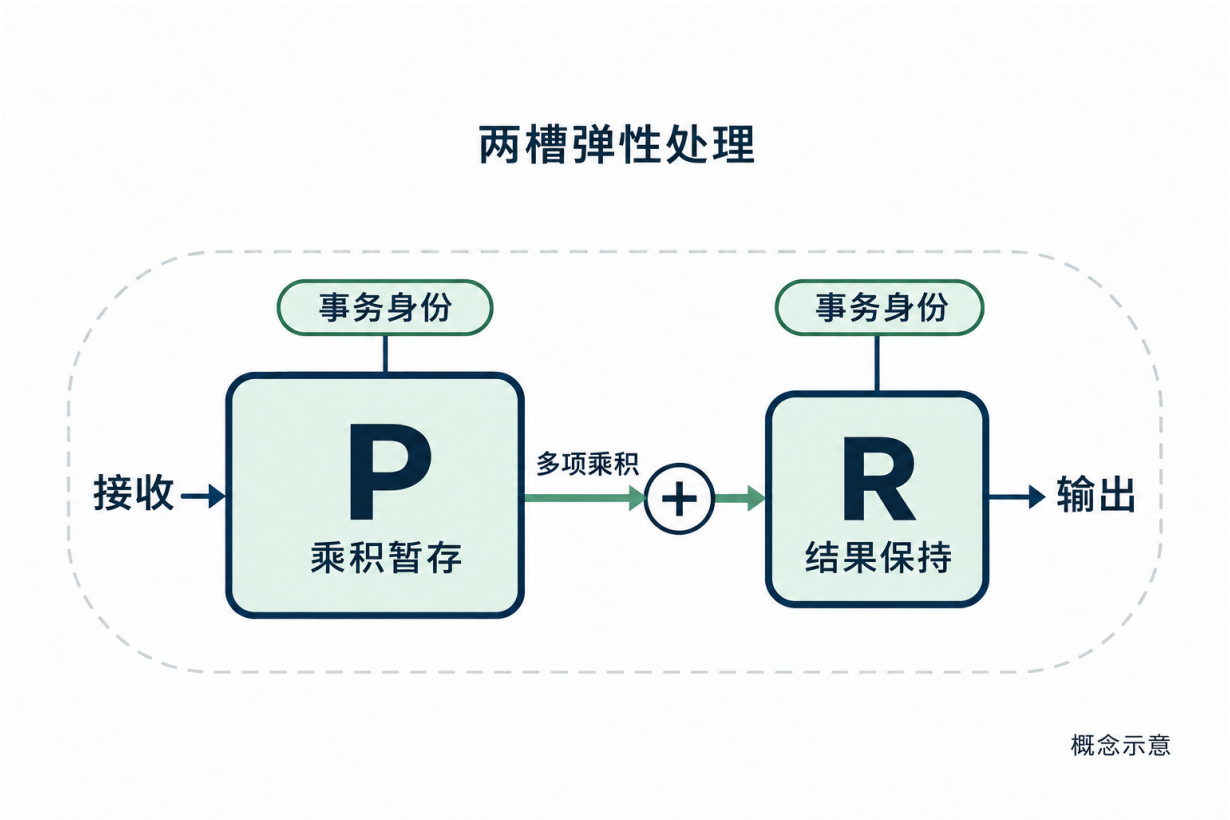
例如 `vdot.vv v10,v8,v9` 对应 `0x0284850b`。程序通过经过校验的编码宏或机器字生成该指令。

向量功能启用、vill=0、vm=1 以及本节的 SEW、LMUL、VL 和 vstart 共同确定合法状态，四种 vta/vma 策略组合均适用。合法执行完成一次完整目的写回，并保持 VL、vtype、vlenb、vxrm、vxsat 和 vcsr。非法请求携带 illegal-instruction 异常完成，同时关闭向量与 v0 目的写使能，由处理器的精确异常机制处理。

2.3 数据通路与弹性两槽控制

2.3.1 乘积槽与结果槽

点积数据通路包含 8 个有符号 8×8 乘法器、乘积槽 P、归约网络和结果槽 R。P 保存 8 个 16 位乘积及这笔请求的控制、性能信息；R 保存 32 位结果及同一请求的身份。结果从 R 输出时补入 96 位零，兼顾紧凑的内部状态和完整的向量写回接口。



概念示意：乘积暂存与结果保持。技术依据见图 2-2。

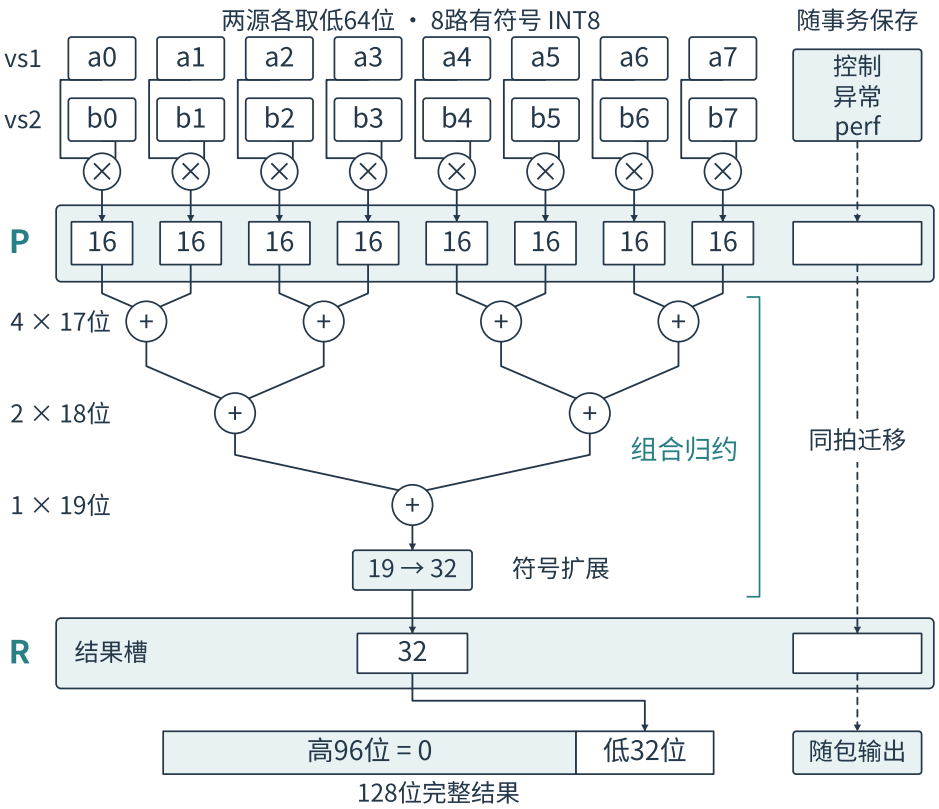


图 2-2　八项点积的乘积槽、归约网络和结果槽

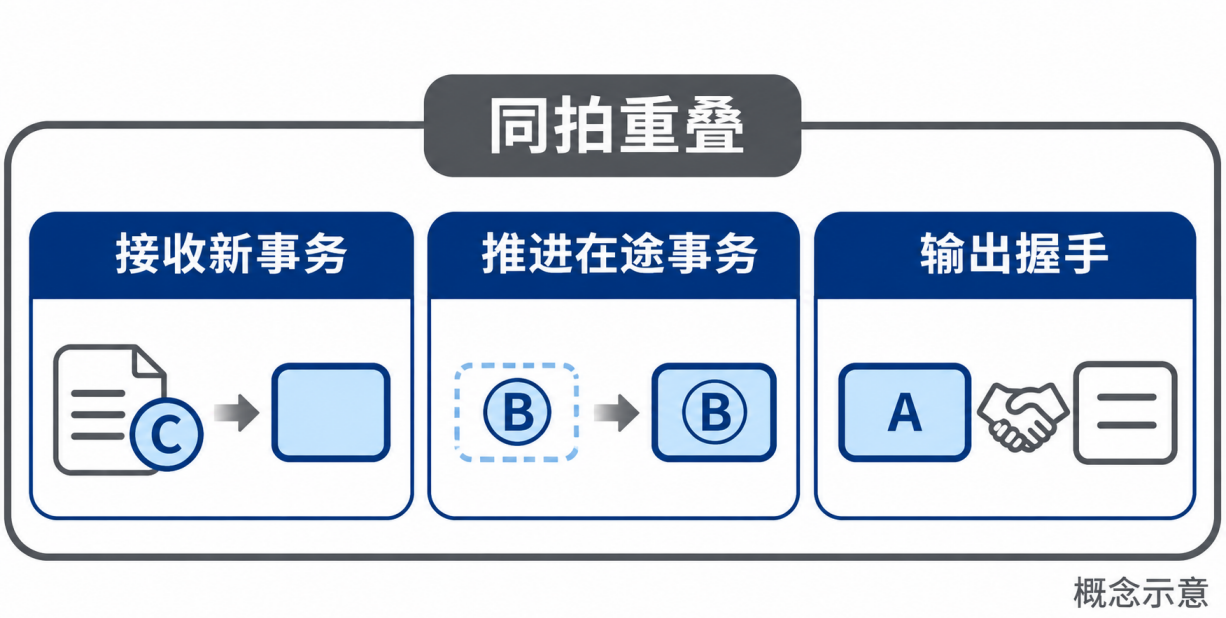
P 保存 8 路 16 位乘积，R 保存 32 位结果；控制与目的身份随数据流动，输出接口为 128 位。

两槽的有效位表示各自是否保存一笔请求。重排序缓冲区（ROB）编号、目的物理寄存器、向量或 v0 写使能、异常信息以及性能字段与数据同时移动，使写回始终能够找到正确的目的。输出停顿时，新输入的变化与 R 中已有请求相互独立。

2.3.2 时钟、复位与连续流动

VDot 功能单元（FU）接入共享执行单元 VFEX4，后文将 VFEX4 称为父级。实现沿用功能单元所在的单时钟域和父级复位连接。P、R 在时钟上升沿更新；复位清除两槽有效位，并抑制复位期间的输入和输出握手。在父级选择并允许接收 VDot、独立请求连续供给、下游持续就绪且没有复位或重定向时，请求于 E0 进入 P，E1 完成归约并进入 R，E2 是最早的输出握手沿。连续请求的发起间隔为 1 拍。

P 状态	R 状态	下游状态	本拍动作
空	空	任意	接受新请求到 P
有效	空	任意	原 P 进入 R，同时可接收下一请求
空	有效	反压	保持 R，P 可接收一笔请求
有效	有效	反压	保持两槽，暂停接收
有效	有效	就绪	原 R 输出、原 P 进入 R、新请求进入 P



概念示意：接收、推进与输出的重叠。技术依据见图 2-3。

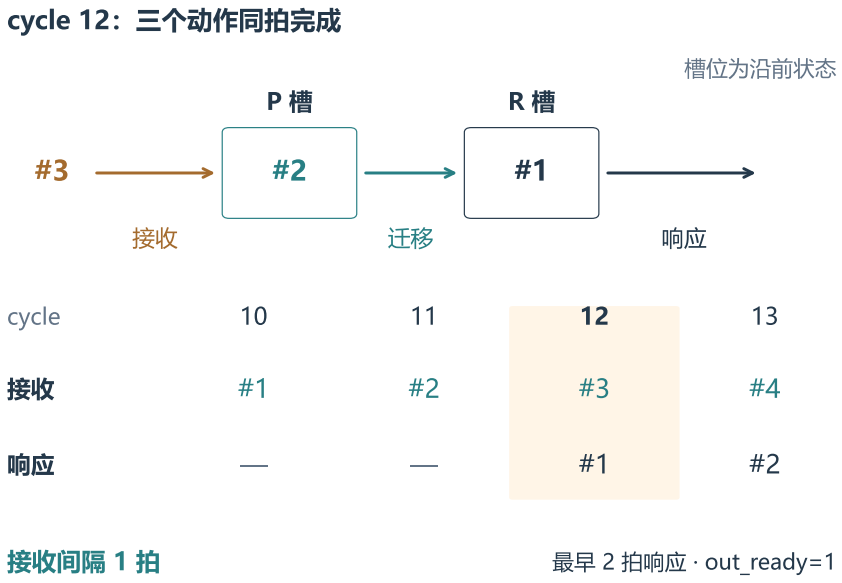


图 2-3　连续请求在两槽中的逐拍流动

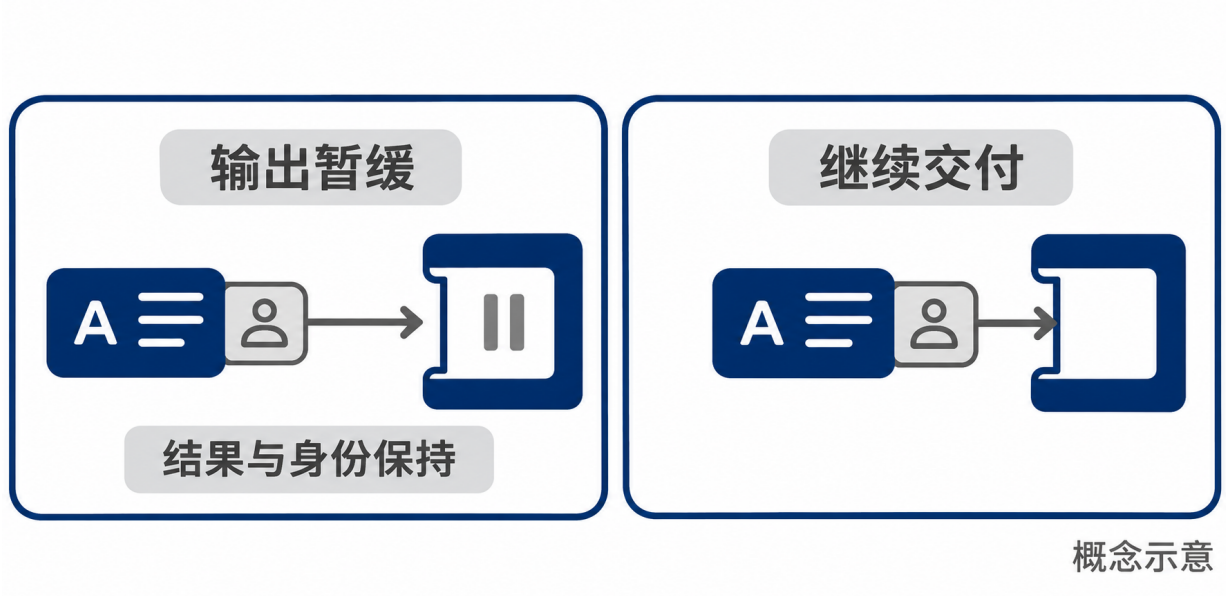
持续供给且下游就绪时，旧 R 输出、旧 P 推进和新请求接收可以在同一拍重叠；最早响应为接受后 2 拍，发起间隔为 1 拍。

子单元就绪与父级准入分别判定。槽位腾出提供接收容量，父级仍根据当前选择与共享资源占用决定何时派发新请求。

时延描述一笔请求从接受到响应的距离，发起间隔描述相邻请求的接受节奏。两槽结构支持连续独立点积；程序的最终周期还由数据依赖、调度、访存和写回共同决定。功能单元继续通过 valid/ready 报告完成，反压时可自然延后响应。

2.3.3　背压、重定向与事务守恒

R 有效而下游未就绪时，数据、目的和控制包一起保持。P 可利用空槽再接收一笔请求；若两槽均占用且 R 仍受阻，输入 ready 撤销，直到 R 输出或取消腾出空间。



概念示意：等待期间保持事务。技术依据见图 2-4。

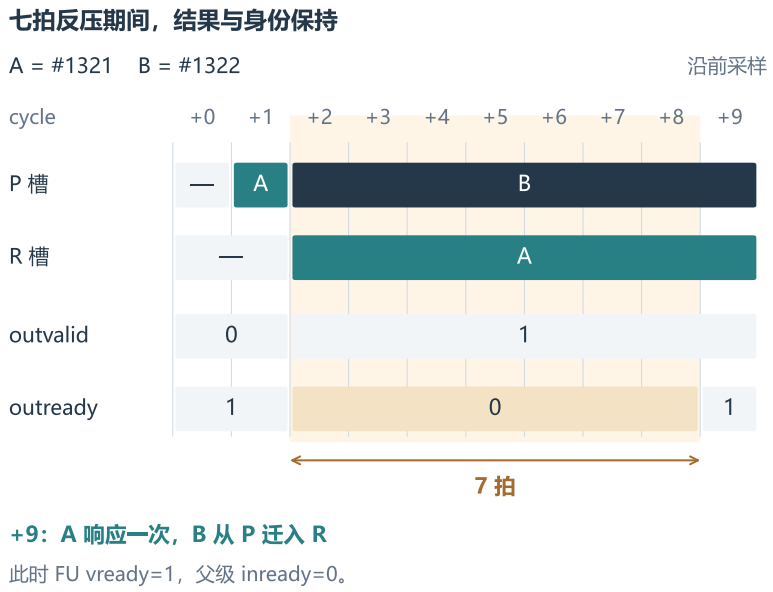


图 2-4 背压期间的数据与目的身份保持

R 在输出阻塞期间保存完整响应；空闲 P 可继续接收一笔请求，满槽后向上游施加反压。

重定向按两个槽各自的 ROB 年龄独立判断取消，保留较老的幸存请求，并在该拍暂停新请求接收。被取消的 R 停止输出，被取消的 P 释放待归约状态；复位的优先级最高。ROB 环回场景使用完整指针年龄关系确定先后。

概念示意：取消与复位的处理边界。技术依据见图 2-5。

图 2-5 部分取消、全部取消与复位恢复

分别判断 P、R 中的请求是否被重定向命中，展示幸存请求、取消清空和复位后重新接收。

每次验证在统一时钟坐标下检查

$$N_{\text{accept}} = N_{\text{response}} + N_{\text{cancel}} + N_{\text{reset}} + N_{\text{inflight}}, \qquad 0 \le N_{\text{inflight}} \le 2.$$

该关系将每笔已接受请求归入响应、取消、复位丢弃或当前占用之一，并在运行结束时检查两槽排空。每笔响应还核对完整 128 位结果、ROB 和目的身份。

概念示意：保留、取消与继续处理。技术依据见图 2-6。

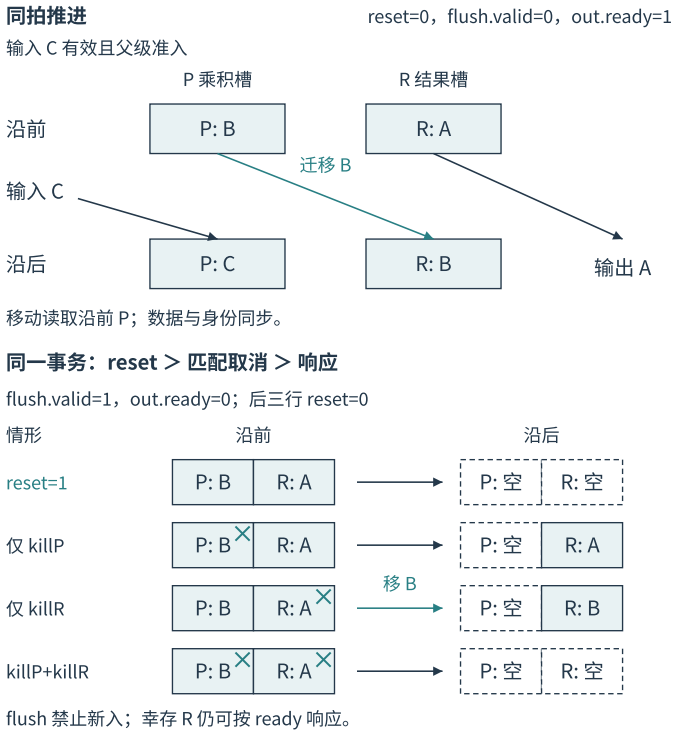


图 2-6 两槽同拍推进与取消、复位处理

满槽时原 R 输出、原 P 推进与新请求接受可在同一拍进行；取消与复位示例说明被撤销请求和幸存请求的处理。

2.4 RTL 实现与处理器接入

2.4.1 关键实现语句

算术部分位于 src/main/scala/xiangshan/backend/fu/vector/VDotDatapath.scala，乘积和每级归约按有符号类型计算，并逐级保留进位。wrapper/VDotElastic.scala 保存 P、R 的控制和数据。下面摘示从 P 转入 R 的关键语句：

```
when(move) {
  rPacket.ctrl := pCtrl
  rPacket.perfDebugInfo := pPerf
  rPacket.res.data :=
  Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
    0.U(32.W), reduce.io.result(31, 0))
}
```

这些赋值读取本拍边沿前的 P 内容，因此同拍接收的新请求会进入 P，原请求的结果和身份则进入 R。公开输出使用：

```
io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

结果寄存器的保存位宽为 32 位，高 96 位由输出组合逻辑补零。

2.4.2 译码、配置依赖与写回

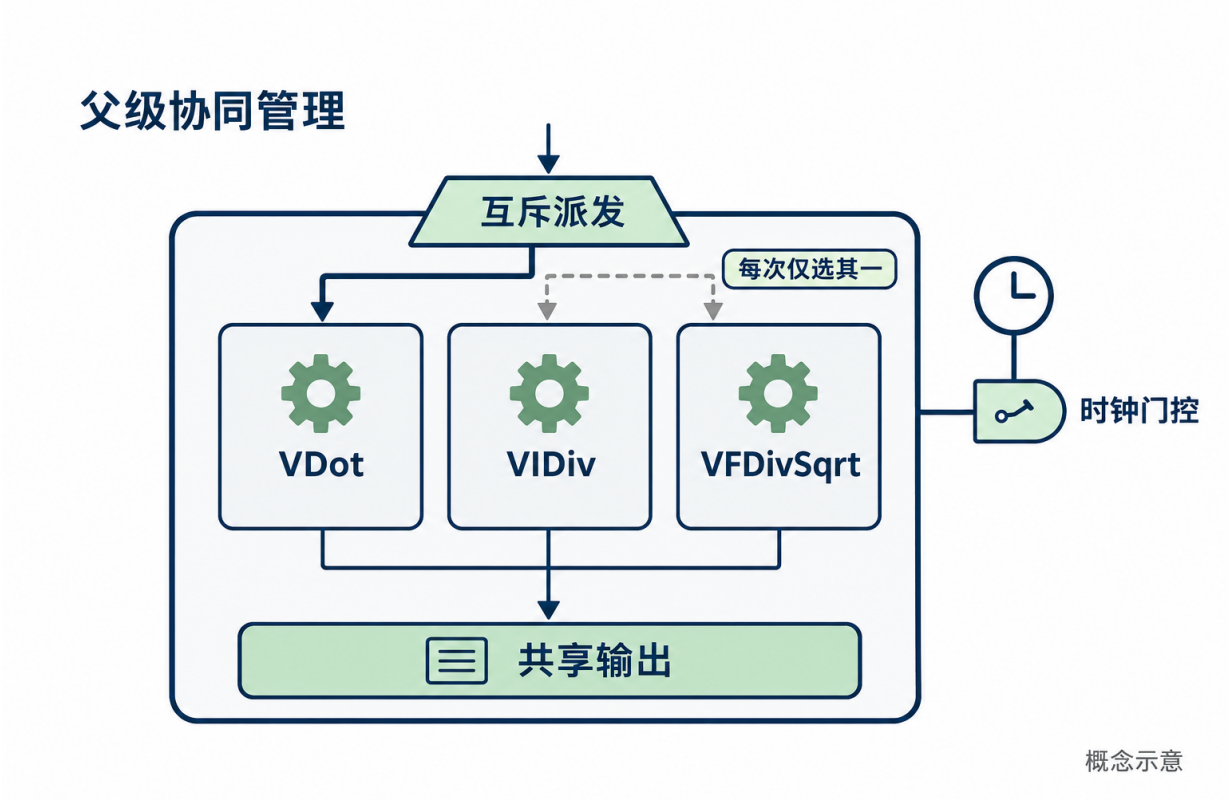
精确编码匹配后，译码器将 vdot 归入向量运算，生成单个微操作并建立两个向量源依赖。实际 VL 通过重命名后的依赖传递，SEW、LMUL 和其他向量控制随指令保存。这样，多条向量配

置与运算交错时，每条点积仍使用与自己对应的配置。

旧目的值不参与累加，译码去除相应的旧目的依赖；向量和 v0 的目的分配、异常信息及最终写回沿用现有处理器机制。选择两槽构建时显式启用 `VDOT_O2_ELASTIC=1` 和 `VDOT_T14_FUSED_INPUT=1`，并采用对应配置清单。工程同时保留通用单请求路径，便于按构建版本比较和回退。

2.4.3 共享 VFEX4 的派发与门控

VDot 与整数 DIV、浮点 DIV/SQRT 共用 VFEX4 父级。父级根据选择、实际占用和被选功能单元的 ready 进行原子派发，使父级和子单元接受的是同一笔请求。VDot 两槽排空后可以切换到 DIV；DIV 在途期间保存自己的 ROB 身份和占用状态。输出按唯一有效完成来源选择共享写回。



概念示意：VFEX4 父级派发与共享输出管理。技术依据见图 2-7。

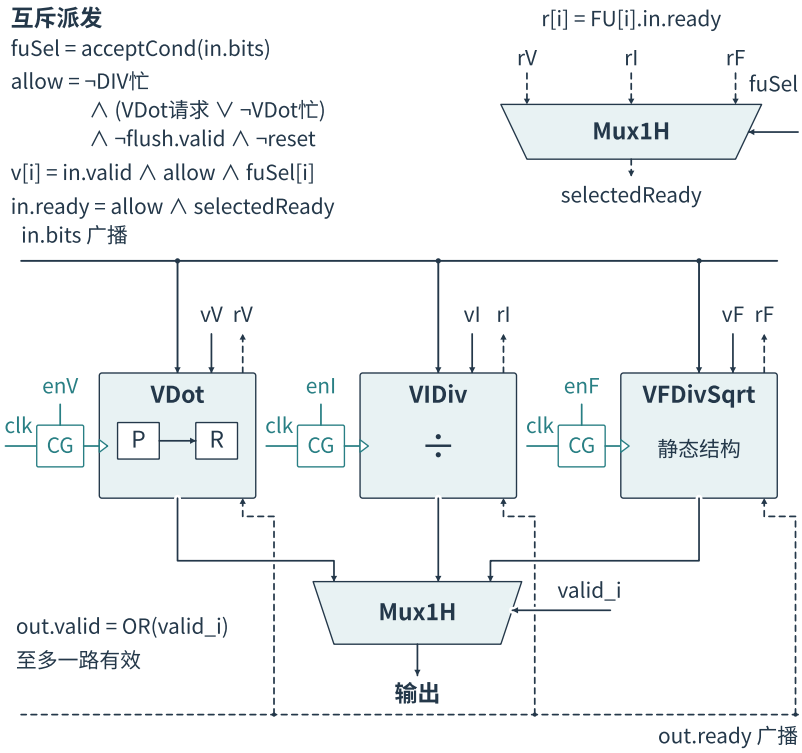
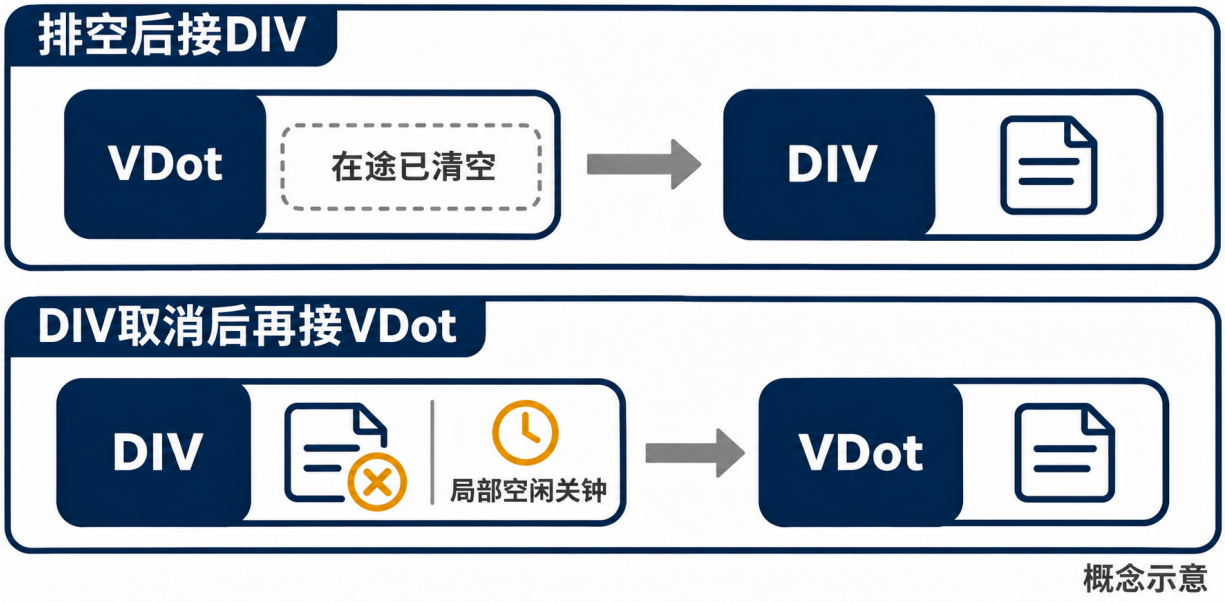


图 2-7 VFEX4 中的派发、资源互斥与共享写回

VDot、整数 DIV 和浮点 DIV/SQRT 共享父级；各自的真实占用决定派发和时钟保持。

时钟使能覆盖存活请求、当前输入接受和必要的取消、复位清理。对应验证包含满槽背压、部分取消、ROB 环回、关钟后的复位恢复，以及 VDot 和两类 DIV 之间的切换。受测的 DIV 取消修复在匹配请求撤销后释放时钟保持，使父级能够进入空闲并接受后续工作。



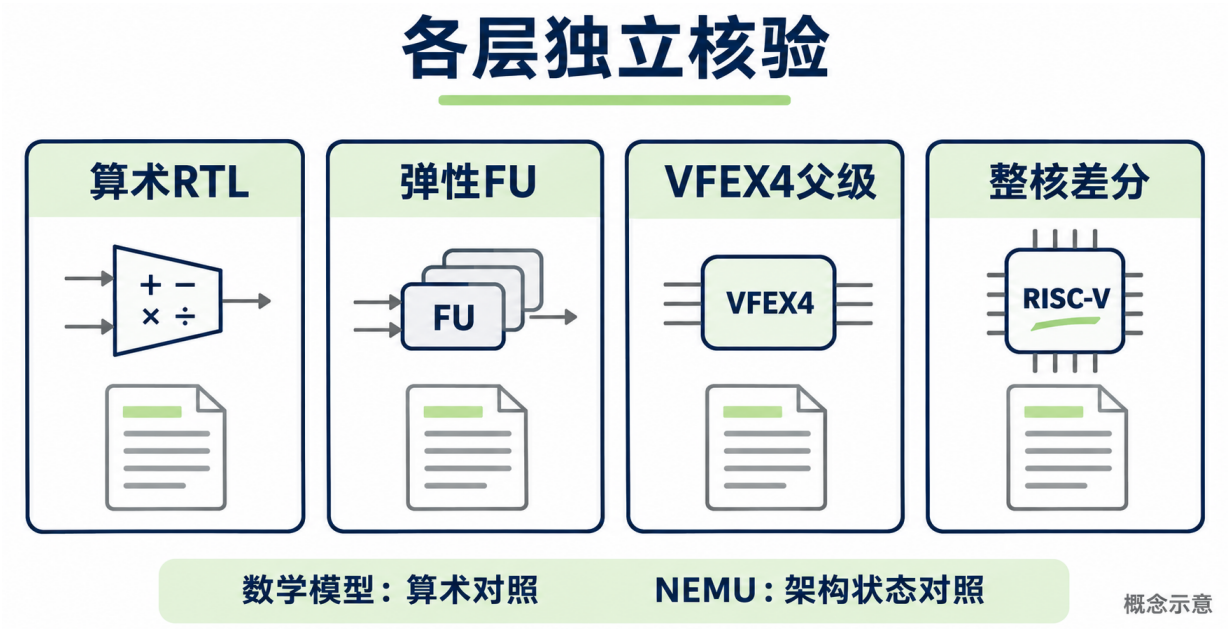
概念示意：功能切换与时钟管理。技术依据见图 2-8。

图 2-8　共享资源切换、取消与时钟保持

按存活请求保持时钟，在取消或完成后释放相应占用，随后恢复派发。

2.5　基础功能与分层验证

验证首先用独立数学计算确定完整 128 位结果，再依次检查算术 RTL、带握手的功能单元、真实父级和全核执行。算术测试关注有符号乘法、极值和逐级位宽；FU 与父级测试加入背压、取消、复位、目的身份和 DIV 交互；整核协同验证进一步检查译码、架构状态及提交结果。



概念示意：不同层级的验证视角。技术依据见图 2-9。

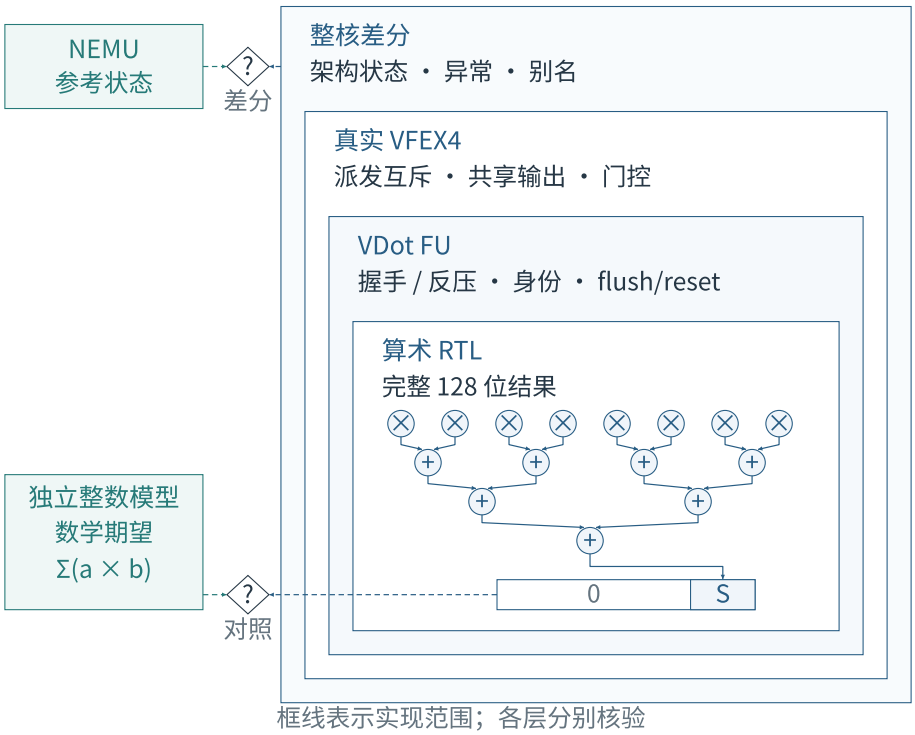


图 2-9 从数学结果到整核执行的验证层次

模型、算术 RTL、FU、真实 VFEX4 父级和整核各检查对应机制；完整 128 位结果贯穿各层。

以下三类测试对应赛题 2.3.1 至 2.3.3。图中的输入与响应取自固定八项两槽版本的全核观察记录，按 seq、ROB 和目的物理寄存器配对；第三阶段说明该版本的架构差分验证。

2.5.1 正数点积测试

两组有效输入均为 $[1, 2, 3, 4, 5, 6, 7, 8]$，八项平方和为 204。全核观察记录 `functional-2` 中，实际完整 128 位结果为 0x000000000000000000000000000000cc，与预期一致。源高 64 位填入 A5 哨兵，用于检查硬件是否准确选择了低 8 个元素。

```
[POSITIVE] functional-2  seq=1  ROB=0:39  pdest=103
vs1 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
vs2 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
expected sum      204
expected (128-bit) 0x000000000000000000000000000000cc
actual   (128-bit) 0x000000000000000000000000000000cc
ACCEPT cycle=339296 line=7030 | RESPONSE cycle=339298 line=7034
source: clean-02/fullcore-run-01/runs/functional-2/run.log
SHA256: 90b36b19e9d141e1b0bdad6bed461b4dbde4e534798f5fbdaa8819d9e2ef3211
```

图 2-10 正数点积：预期与实际结果均为 204

两个源的有效元素为 1 至 8，响应低 32 位为 `000000cc`，高 96 位为零。

2.5.2 负数点积测试

有效输入为八个 -1 与八个 1，预期点积为 -8。`functional-0` 记录的完整结果为 0x0000000000000000000000000000fffffff8。低 32 位以二补码表达负数，高 96 位保持零，符合指令规定的结

果布局。

```
[NEGATIVE] functional-0  seq=4  ROB=0:12  pdest=143
vs1 (128-bit)      0xa5a5a5a5a5a5a5a5ffffffffffffffff
vs2 (128-bit)      0xa5a5a5a5a5a5a5a501010101010101010
expected sum       -8
expected (128-bit)  0x000000000000000000000000fffffff8
actual   (128-bit)  0x000000000000000000000000fffffff8
ACCEPT cycle=420291 line=7927 | RESPONSE cycle=420293 line=7931
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-11　负数点积：低 32 位保存负数补码

八个 −1 与八个 1 相乘求和；实际低 32 位为 `fffffff8`，高 96 位为零。

2.5.3　边界值点积测试

最大正结果由八个 −128 与八个 −128 产生。每项乘积为 16384，总和为 131072；`functional-0` 记录的完整结果为 0x00000000000000000000000000020000。这一输入检验了乘法和归约在正端点的精确性。

```
[BOUNDARY] functional-0  seq=5  ROB=0:43  pdest=66
vs1 (128-bit)      0xa5a5a5a5a5a5a5a580808080808080808
vs2 (128-bit)      0xa5a5a5a5a5a5a5a580808080808080808
expected sum       131072
expected (128-bit)  0x00000000000000000000000000020000
actual   (128-bit)  0x00000000000000000000000000020000
ACCEPT cycle=448417 line=8400 | RESPONSE cycle=448419 line=8404
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-12　正端点：八对最小 INT8 数产生 131072

实际低 32 位为 `00020000`，高 96 位为零，与八项和的正端点一致。

零输入测试采用八个 0 与八个 127，实际完整 128 位响应为零。另一数学边界由八个 −128 与八个 127 构成，预期总和为 −130048、低 32 位为 `0xfffe0400`；该值作为负端点的参考判据列入定向测试定义。

2.5.4　算术数据通路测试汇总

算术 RTL 测试包含 22 项定向输入和 10000 项随机输入，共 10022 项全部通过。定向输入针对符号、零值、极值和元素选择，随机输入进一步扩大数值组合覆盖。图中汇总描述算术数据通路；带背压和取消的时序行为在 FU 及父级测试中检查。

```
VDOT_DATAPATH_SUITE_PASS suite=directed count=22
VDOT_DATAPATH_SUITE_PASS suite=random count=10000
VDOT_DATAPATH_PASS directed=22 random=10000 total=10022
```

图 2-13 算术 RTL 的定向与随机测试汇总

22 项定向测试与 10000 项随机测试，共 10022 项全部通过。

2.5.5 响应配对与复查位置

四类已展示响应均按 seq、ROB 和 pdest 与接受记录配对，并比较完整 128 位结果。具体位置如下，可与原始图片中的输入和响应互相核对。

输入类型	日志	接受/响应周期	原始行号
正数	`functional-2/run.log`	339296 / 339298	7030–7034
负数	`functional-0/run.log`	420291 / 420293	7927–7931
最大正结果	`functional-0/run.log`	448417 / 448419	8400–8404
零输入	`functional-0/run.log`	311084 / 311086	5814–5818

零输入对应 seq=0、ROB=0:8、pdest=124。原始记录位于固定八项两槽受测版本的 clean-02/fullcore-run-01/runs/归档；本节展示执行单元响应，完整运行的架构差分结果见第三阶段。

(a) Historical combinational RTL suite counts (2026-10-12)

```
VDOT_DATAPATH_SUITE_PASS suite=directed count=22
VDOT_DATAPATH_SUITE_PASS suite=random count=10000
VDOT_DATAPATH_PASS directed=22 random=10000 total=10022
```

(b) Positive transaction: expected 204 and actual 128-bit response

```
[POSITIVE] functional-2 seq=1 ROB=0:39 pdest=103
vs1 (128-bit)      0xa5a5a5a5a5a5a5a50807060504030201
vs2 (128-bit)      0xa5a5a5a5a5a5a5a50807060504030201
expected sum       204
expected (128-bit) 0x000000000000000000000000000000cc
actual   (128-bit) 0x000000000000000000000000000000cc
ACCEPT cycle=339296 line=7030 | RESPONSE cycle=339298 line=7034
source: clean-02/fullcore-run-01/runs/functional-2/run.log
SHA256: 90b36b19e9d141e1b0bdad6bed461b4dbde4e534798f5fbdaa8819d9e2ef3211
```

(c) Negative transaction: expected -8 and actual 128-bit response

```
[NEGATIVE] functional-0 seq=4 ROB=0:12 pdest=143
vs1 (128-bit)      0xa5a5a5a5a5a5a5a5ffffffffffffffff
vs2 (128-bit)      0xa5a5a5a5a5a5a5a50101010101010101
expected sum       -8
expected (128-bit) 0x00000000000000000000000ffffff8
actual   (128-bit) 0x00000000000000000000000ffffff8
ACCEPT cycle=420291 line=7927 | RESPONSE cycle=420293 line=7931
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

(d) Boundary transaction: expected 131072 and actual 128-bit response

```
[BOUNDARY] functional-0 seq=5 ROB=0:43 pdest=66
vs1 (128-bit)      0xa5a5a5a5a5a5a5a58080808080808080
vs2 (128-bit)      0xa5a5a5a5a5a5a5a58080808080808080
expected sum       131072
expected (128-bit) 0x00000000000000000000000020000
actual   (128-bit) 0x00000000000000000000000020000
ACCEPT cycle=448417 line=8400 | RESPONSE cycle=448419 line=8404
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-14　正数、负数与边界测试的输入和结果

集中对照正数 204、负数 −8 和最大正结果 131072，观察低 32 位值及高 96 位零；算术统计与前述汇总为同一组数据。

在基础数值之外，测试还检查源高半区哨兵、旧目的哨兵、寄存器别名、v0 目的、非法配置和向量状态恢复。FU 与真实父级通过响应身份、写使能、取消和高位错误注入检查判定链，并在正常运行中核对完整响应与排空。

2.6　设计取舍与扩展

2.6.1　两槽结构与结果保存

两槽结构使乘法和归约能够与相邻请求重叠，面向连续独立点积提高供给能力。将 R 中的结果保存宽度收窄至 32 位，利用了高 96 位固定为零这一架构性质，减少了数据寄存器成本。每笔请求的 ROB、目的和控制信息则完整保留，以支持乱序执行、背压和精确取消。

共享父级的优化同时考虑正常完成与取消后的资源释放。方案选择综合功能、周期、标准单元面积和时序路径：一拍算术候选在成本筛选中暴露的父级代价，支持继续使用已有两槽实现作为对应性能表的受测对象。面积和各类路径的具体数据集中列在第三阶段。

2.6.2　十六项 INT8 点积的功能进展

在八项实现的基础上，团队增加了独立十六项点积候选。该候选以补丁和配套验证快照交付，采用 `MATCH=0x0e00000b`、`MASK=0xfe00707f`，在 e8/m1、VL=16 时消费两个 128 位源的

全部 16 个字节。八项指令继续使用原编码和 VL=8 语义。

候选复用两组八项乘积及归约模块。P 保存 16 个 16 位乘积，两组 19 位部分和相加得到 20 位精确和，再扩展为低 32 位结果。十六项和的范围为 $[-260096, 262144]$，R 仍保存 32 位，写回高 96 位为零。接收八项指令时，高八个乘积写零，使后级统一完成归约。两槽握手和取消机制沿用原结构。

2026 年 11 月 6 日，该候选完成了编译开关关闭和开启两种配置下的译码、FU 和真实父级验证。启用十六项扩展后，使用新 EMU 与匹配 NEMU 完成六例整核差分：2 例八项指令、4 例十六项指令全部通过。六例覆盖源高半区贡献、有符号正负端点、选定源目的别名、完整 128 位写回和恢复八项指令。相应 FU 与父级连续请求场景还核对了 2 拍首响应与 1 拍发起间隔。

该扩展的当前成果是指令与执行通路的功能验证。后续将通过完整 Q288 调用、与较快 RVV 实现的性能对照，以及同条件下的综合与时序分析，评价其应用价值。本报告的主要周期和成本表对应表中命名的八项受测版本。

2.6.3 元素位宽的参数化验证

另一组独立模块实验把输入元素扩展为 INT16，计算 8 项点积，并使用低 64 位结果字段、高 64 位清零的输出布局。该配置的最大正和为 2^{33}，精确累加至少需要 35 位。参数化乘法与归约模块完成 94 项定向输入和 10000 项随机输入的完整 128 位结果检查，并验证单请求包装中的反压、取消与复位行为。这组结果展示了从元素位宽到中间精度、结果字段和测试向量的一致扩展方法。

2.7 实现与验证摘录

下列摘录把前文的数据通路、状态控制和验证方法对应到具体实现与运行记录。各图分别展示源码、主机模型和逐拍日志，便于结合正文复查。

(a) Fixed 0f VDotProducts source excerpt

```
class VDotProducts extends RawModule {
  val io = IO(new Bundle {
    val vs1 = Input(UInt(128.W))
    val vs2 = Input(UInt(128.W))
    val products = Output(Vec(8, SInt(16.W)))
  })

  for (lane <- 0 until 8) {
    val a = io.vs1(8 * lane + 7, 8 * lane).asSInt
    val b = io.vs2(8 * lane + 7, 8 * lane).asSInt
    io.products(lane) := a * b
  }
}

/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
```

(b) Fixed 0f VDotReduce source excerpt

```
/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
  val io = IO(new Bundle {
    val products = Input(Vec(8, SInt(16.W)))
    val result = Output(UInt(128.W))
  })

  private val sums17 = Wire(Vec(4, SInt(17.W)))
  private val sums18 = Wire(Vec(2, SInt(18.W)))
  private val sum19 = Wire(SInt(19.W))

  for (pair <- 0 until 4) {
    sums17(pair) := io.products(2 * pair) +& io.products(2 * pair + 1)
  }
  for (pair <- 0 until 2) {
    sums18(pair) := sums17(2 * pair) +& sums17(2 * pair + 1)
  }
  sum19 := sums18(0) +& sums18(1)

  private val result32 = Wire(SInt(32.W))
  result32 := sum19
  io.result := Cat(0.U(96.W), result32.asUInt)
}

/** Combinational arithmetic top for independent verification.
  * The future functional-unit wrapper instantiates the same two submodules
  * separately, with registers between them; this top has no cycle protocol.
  */
```

图 2-15　八项点积算术实现摘录

八路有符号乘法和逐级归约，对应本章的中间位宽分析。

(a) Slot registers and 32-bit result state

```
    require(!cfg.writeFflags && !cfg.writeVxsat && !cfg.flushPipe && !cfg.replayInst)

    val pipelineStatus = IO(Output(new VDotElasticStatus))

>   private val pValid = RegInit(false.B)
    private val rValid = RegInit(false.B)
    private val pProducts = Reg(Vec(8, SInt(16.W)))
    private val pCtrl = Reg(chiselTypeOf(io.out.bits.ctrl))
    private val pPerf = Reg(chiselTypeOf(io.out.bits.perfDebugInfo))
    // H1 stores only the architectural low word; the public result stays 128 bits.
    // Keep each slot's complete control/perf identity on the same transfer edge.
    private val rPacket = Reg(new Bundle {
      val ctrl = chiselTypeOf(io.out.bits.ctrl)
      val res = new Bundle {
        val data = UInt(32.W)
      }
      val perfDebugInfo = chiselTypeOf(io.out.bits.perfDebugInfo)
    })

    pipelineStatus.pValid := pValid
    pipelineStatus.rValid := rValid
    pipelineStatus.occupancy := pValid.asUInt +& rValid.asUInt

    private val illegalInput = vill || vsew =/= VSew.e8 || vlmul =/= VLmul.m1 ||
      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
```

(b) Handshake and cancellation conditions

```
PS C:\WINDOWS\system32> wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -Pattern 'private val rReady
=' -Context 5,12

      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
    private val killP = pValid && pCtrl.robIdx.needFlush(io.flush)
    private val killR = rValid && rPacket.ctrl.robIdx.needFlush(io.flush)
    private val liveP = pValid && !killP && !reset.asBool
    private val liveR = rValid && !killR && !reset.asBool
>   private val rReady = !liveR || io.out.ready
    private val move = liveP && rReady
    private val pReady = !liveP || rReady

    // Parent admission is enforced by its atomic child dispatch. The FU also
    // suppresses every new accept on redirect/reset, including unrelated flush.
    io.in.ready := pReady && !io.flush.valid && !reset.asBool
    io.out.valid := liveR
    io.out.bits.ctrl := rPacket.ctrl
    io.out.bits.perfDebugInfo := rPacket.perfDebugInfo
    io.out.bits.res := 0.U.asTypeOf(io.out.bits.res)
    io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

(c) State update and old-P transfer

```
PS C:\WINDOWS\system32>
                       > wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -SimpleMatch 'pValid := (live
P && !move) || io.in.fire' -Context 2,16

    acceptedCtrl.exceptionVec.get(ExceptionNO.illegalInstr) := illegalInput

>   pValid := (liveP && !move) || io.in.fire
    rValid := (liveR && !io.out.fire) || move

    when(io.in.fire) {
      pProducts := products.io.products
      pCtrl := acceptedCtrl
      pPerf := io.in.bits.perfDebugInfo
    }
    when(move) {
      // Reads pre-edge P even when a new request replaces it at this same edge.
      rPacket.ctrl := pCtrl
      rPacket.perfDebugInfo := pPerf
      rPacket.res.data := Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
        0.U(32.W), reduce.io.result(31, 0))
    }

    // Read-only handles for the separate G1/G2 observation harness. These carry
```

图 2-16　两槽状态与控制实现摘录

结果槽保存 32 位；有效位、握手、取消条件和状态更新共同管理请求。

Host model checks only; no RTL/NEMU execution in this command

```
HELLO_VADD_EMU_RUN_PASS
PS C:\WINDOWS\system32> wsl -d Ubuntu-22.04 -- bash /home/riscv/riscv-lab/zjhwork2-review-20261018/repo/team/scripts/check-vdot-model.sh
test_check_detects_corruption_and_leaves_every_file_unchanged (test_generate.GeneratorTests) ... ok
test_check_missing_output_does_not_create_directory (test_generate.GeneratorTests) ... ok
test_check_without_python_B_is_readonly_and_detects_source_drift (test_generate.GeneratorTests) ... ok
test_directed_manual_expectations_and_formats (test_generate.GeneratorTests) ... ok
test_manifest_hashes_and_smoke_selection (test_generate.GeneratorTests) ... ok
test_prng_contract_byte_order_and_repeatability (test_generate.GeneratorTests) ... ok
test_state_aliases_use_one_real_prestate (test_generate.GeneratorTests) ... ok
test_10000_fixed_seed_pairs_with_independent_byte_oracle (test_model.ArithmeticTests) ... ok
test_D00_through_D10 (test_model.ArithmeticTests) ... ok
test_all_65536_single_signed_byte_products (test_model.ArithmeticTests) ... ok
test_contract_identity_and_dimensions (test_model.ArithmeticTests) ... ok
test_extrema_require_19_signed_bits (test_model.ArithmeticTests) ... ok
test_high_half_is_ignored_and_negative_is_not_128bit_sign_extended (test_model.ArithmeticTests) ... ok
test_lane_order_and_one_hot_inputs (test_model.ArithmeticTests) ... ok
test_all_32768_register_encodings (test_model.EncodingTests) ... ok
test_every_fixed_bit_flip_is_illegal (test_model.EncodingTests) ... ok
test_known_word_and_source_field_order (test_model.EncodingTests) ... ok
test_random_nonmatching_words_are_illegal (test_model.EncodingTests) ... ok
test_256_random_register_banks_against_byte_oracle (test_model.ExecutionTests) ... ok
test_aliases_and_v0_v31_use_instruction_before_state (test_model.ExecutionTests) ... ok
test_config_is_frozen_and_default_execution_works (test_model.ExecutionTests) ... ok
test_directed_values_with_dirty_old_destination (test_model.ExecutionTests) ... ok
test_illegal_config_has_cause_two_and_no_input_changes (test_model.ExecutionTests) ... ok
test_illegal_encoding_preserves_inputs (test_model.ExecutionTests) ... ok
test_other_vlen_is_unsupported_configuration (test_model.ExecutionTests) ... ok
test_policy_combinations_rounding_saturation_and_vs_dirty (test_model.ExecutionTests) ... ok
test_config_api_types_and_small_field_ranges (test_model.InvalidApiTests) ... ok
test_instruction_input_ranges_and_types (test_model.InvalidApiTests) ... ok
test_lane_inputs_require_exact_length_and_signed_integers (test_model.InvalidApiTests) ... ok
test_packed_sources_and_high_half_require_unsigned_integer_domains (test_model.InvalidApiTests) ... ok
test_register_bank_domains (test_model.InvalidApiTests) ... ok
test_every_serialized_arithmetic_expectation (test_vectors.CommittedVectorTests) ... ok
test_fixed_prng_and_smoke_selection (test_vectors.CommittedVectorTests) ... ok
test_manifest_and_all_file_hashes (test_vectors.CommittedVectorTests) ... ok
test_state_expectations_and_decoder_scope (test_vectors.CommittedVectorTests) ... ok

----------------------------------------------------------------------
Ran 35 tests in 1.353s

OK
OK: 5 files match regenerated bytes in /home/riscv/riscv-lab/zjhwork2-review-20261018/repo/team/models/vdot/data
VDOT_MODEL_CHECK_PASS (host model and data only; no RTL/NEMU run)
PS C:\WINDOWS\system32>
```

图 2-17　独立主机模型的测试输出

35 个主机模型测试方法通过。该统计以测试方法为单位；算术 RTL 的 10022 组输入另见基础测试汇总。

Historical 0f observer log excerpt; NOT a GTKWave waveform

```
S2-05 SUPPORTING EVIDENCE: VDOT TWO-SLOT EVENT LOG EXCERPT
Historical 0f clean-02, dense-batch128, 2026-10-27 (UTC).
Observer event log excerpt; this is NOT a waveform screenshot.
Selected fields copied from the source lines listed below; fields omitted.
Source: /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/dense-batch128/run.log
Source SHA256: 18f472d75feef0378bec71c64793ffd2297bc37c47a5b501597f222da174b98f

line 146 cycle=10984 O2_STATE
  pvalid=0 pseq=0 rvalid=0 rseq=0 occupancy=0 invalid=1 inready=1 outvalid=0 outready=1 move=0
line 148 cycle=10984 O2_ACCEPT
  seq=0 rob=1:31 pdest=110 vl=8 illegal=0 vecwen=1
line 149 cycle=10985 O2_STATE
  pvalid=1 pseq=0 rvalid=0 rseq=0 occupancy=1 invalid=1 inready=1 outvalid=0 outready=1 move=1
line 151 cycle=10985 O2_ACCEPT
  seq=1 rob=1:32 pdest=111 vl=8 illegal=0 vecwen=1
line 152 cycle=10985 O2_MOVE
  seq=0 rob=1:31 pdest=110 illegal=0 vecwen=1
line 153 cycle=10986 O2_STATE
  pvalid=1 pseq=1 rvalid=1 rseq=0 occupancy=2 invalid=1 inready=1 outvalid=1 outready=1 move=1
line 155 cycle=10986 O2_ACCEPT
  seq=2 rob=1:33 pdest=112 vl=8 illegal=0 vecwen=1
line 156 cycle=10986 O2_MOVE
  seq=1 rob=1:32 pdest=111 illegal=0 vecwen=1
line 157 cycle=10986 O2_RESPONSE
  seq=0 rob=1:31 pdest=110 data=0x00000000000000000000000000020000 vecwen=1

At cycle 10986: ACCEPT seq=2, MOVE seq=1, RESPONSE seq=0.
Source result.json reports O2_DENSE_ELASTIC_AND_WINDOWS_PASS and exit_code=0.
Original batch UTC capacity audit FAIL retained; see R5/G5 acceptance RESULT.md.
```

图 2-18　连续事务的同拍重叠记录

10986 周期同时记录 ACCEPT seq2、MOVE seq1 与 RESPONSE seq0，分别对应新输入、原 P 推进和原 R 响应。

2.8 源码与证据入口

内容	阅读入口
指令语义与编码	指令规格与定向向量
算术及两槽实现	八项弹性 FU 源码及乘积与归约源码
构建与版本选择	工程版本入口与可选配置
周期、面积与 STA	八项两槽受测版本的数据与原始报告
十六项扩展	候选结构、补丁与分层验证、六例整核差分结果
元素位宽参数化	八项 INT16 独立模块及完整结果验证

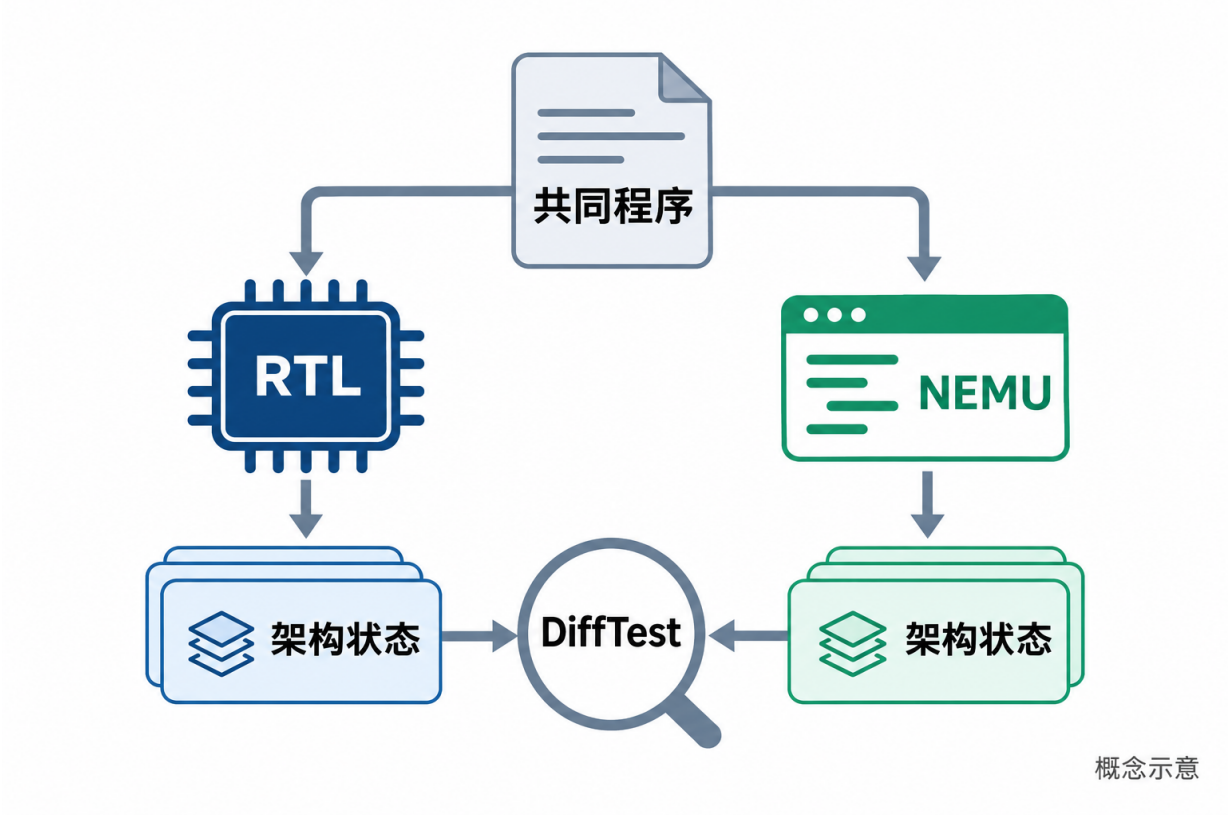
工程索引保留完整源码版本、运行输入、原始日志和失败记录。正文以设计机制和结果解释为主，读者可按需要沿上述入口复查实现细节与测量依据。

第三阶段：协同仿真与性能评估

3.1 从功能验证到工程评价

自定义指令需要在程序中产生正确结果，也需要与处理器的异常、状态和调度机制配合。本阶段使用独立 NEMU 参考模型和全核 DiffTest 核对架构行为，再以等价 RVV 程序比较完整测量窗口的周期。软件接入、综合面积和静态时序共同说明这项设计在不同场景下的收益与成本。

报告分别给出固定八项点积实现的微基准、C8 软件调用和正在推进的完整前向计算。每组结果均说明受测版本、输入与测量边界，便于从局部指令机制逐步理解其应用价值。



概念示意：独立求值与架构状态比较。技术依据见图 3-1。

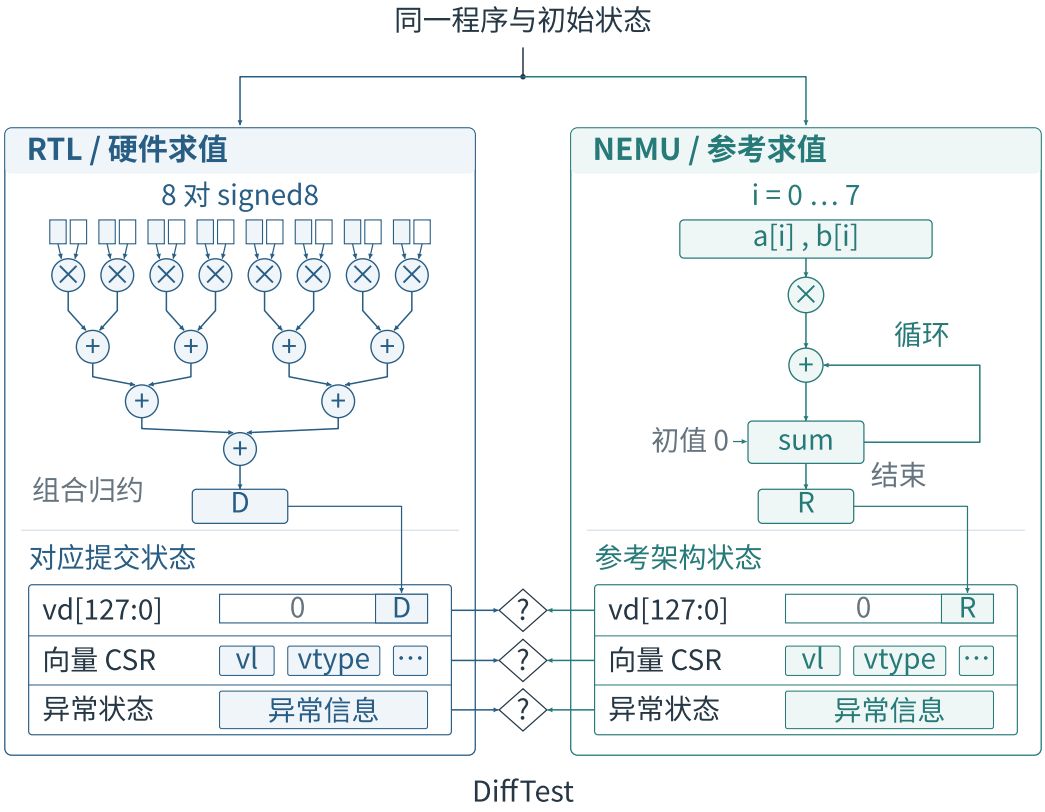


图 3-1 RTL 与独立参考模型的协同仿真

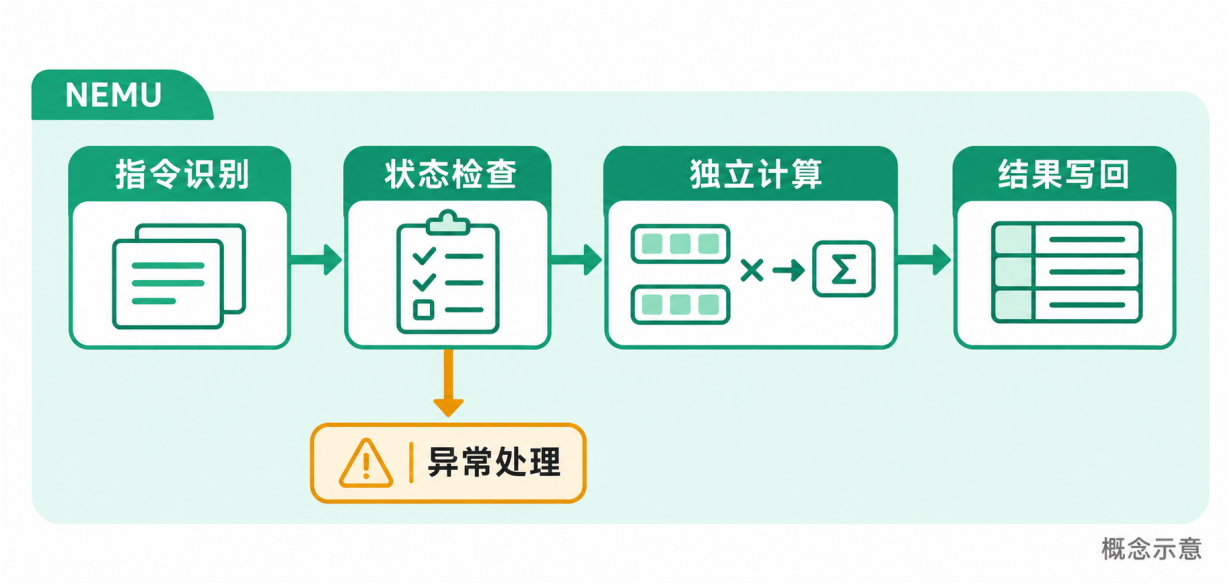
同一程序驱动香山 RTL 与 NEMU 参考模型，在架构提交边界比较向量结果，并按测试约定核对 CSR 和异常状态。

3.2 NEMU 参考语义与处理器接入

RTL 通过并行乘法、归约网络和流水状态执行点积；NEMU 参考模型则从指令字、源寄存器和向量配置出发，顺序计算八项有符号乘积之和。两个实现采用不同内部组织，对同一架构定义形成交叉核对。

参考端首先匹配 custom-0 编码并检查合法配置，随后快照读取两个源寄存器，计算点积并构造完整 128 位目的值：低 32 位为整数结果，高 96 位为零。快照读取使源、目的寄存器重合时仍使用本条指令执行前的数据。合法运算保留约定的向量状态；非法配置进入异常处理。

八项点积受测组合使用基于 NEMU 固定版本的 vdot 补丁及配套平台适配。适配依据指令和架构状态对齐该香山版本的 xTVAL 行为，异常码、陷阱 PC、xTVAL 和目的寄存器状态均保留检查。现有特权测试记录覆盖 M 模式。参考库、故障注入库与独立运行的 NEMU 各自记录构建身份，复现时按用途选择对应制品。



概念示意：识别、检查与执行。技术依据见图 3-2。

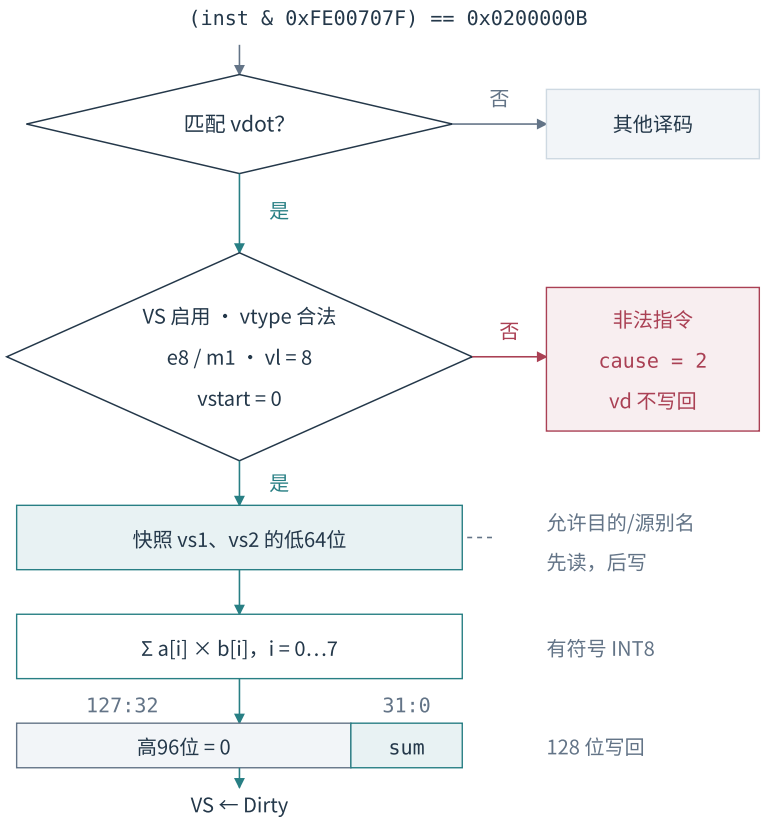


图 3-2　参考模型的译码、状态检查与完整写回

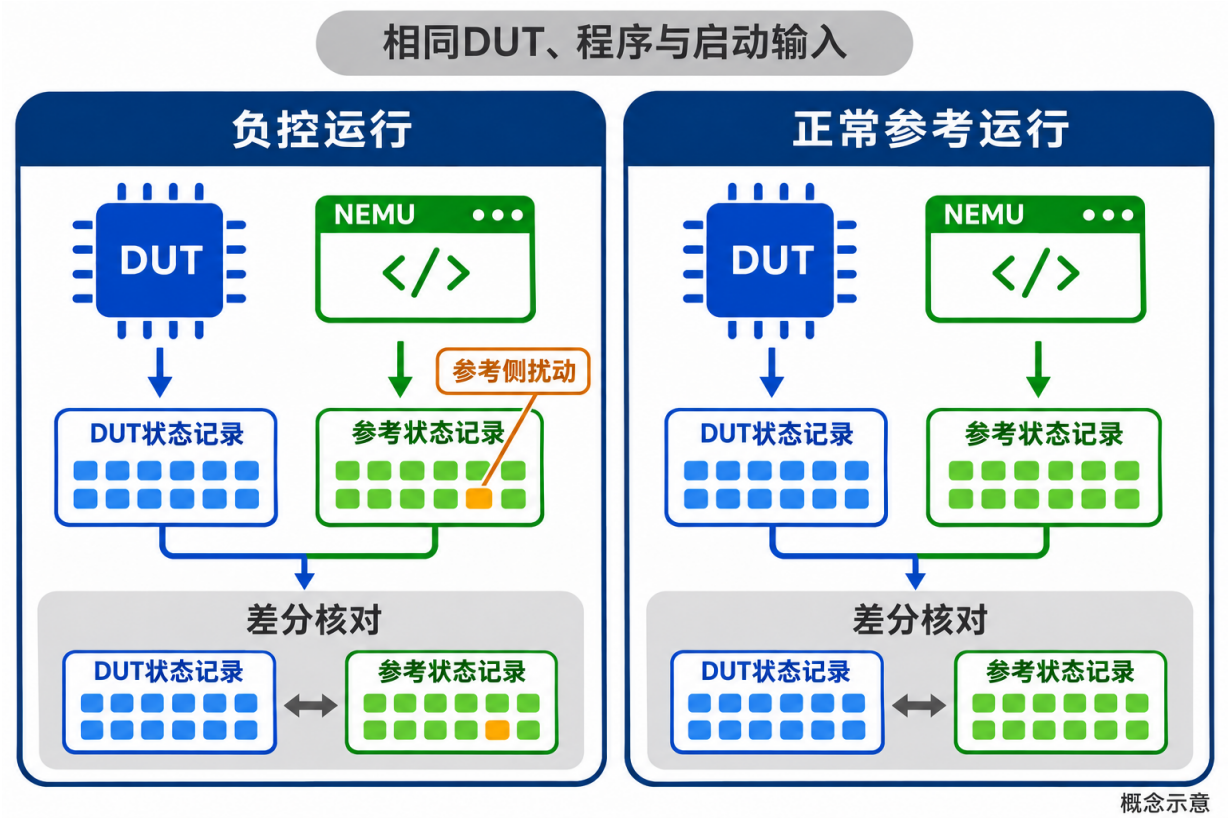
参考端先检查编码与向量状态，再快照源值、求和并写回；非法状态沿异常路径完成。平台适配与对应受测版本共同固定。

3.3　差分检验与错误检出

基础功能矩阵覆盖正数、负数、边界值、寄存器别名和非法向量配置。在八项点积两槽受测组合中，全核矩阵包含 117 个不同用例，按矩阵分片共执行 124 次；算术 RTL 的 10022 项测试和

真实父级控制测试分别承担数值与握手覆盖。R5 组合验收汇合 13 项已归档记录与 1 项新增控制诊断，并重新解析其原始输出。

为了检验完整 128 位比较是否有效，负控在参考端的 vdot 结果中翻转 bit100。差分器检测到目的寄存器高半区不一致并按预期终止；恢复正常参考模型后，同一程序完成 15 项检查并到达 GOOD TRAP。图3-3给出了这两个互补结果：前者检验错误检出能力，后者检验正常执行路径。



概念示意：扰动检查与正常参考对照。技术依据见图 3-3。

（a）参考端翻转 bit100：差分器检出高位差异

```
PS C:\Users\33532> Start-Process 'D:\sharex\ShareX\pages\2026-11\Notepad_kUWl09ct1X.png'
PS C:\Users\33532> wsl -d Ubuntu-22.04 -- tail -n 7 /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/negative-vdot/run.log
privilegeMode: 3
v10_high different at pc = 0x0080001bdc, right= 0x0000001000000000, wrong = 0x0000000000000000
Core 0: ABORT at pc = 0x800016b0
Core-0 instrCnt = 238849, cycleCnt = 311097, IPC = 0.767764
Seed=0 Guest cycle spent: 311100 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 740557ms
PS C:\Users\33532> wsl -d Ubuntu-22.04 -- cat /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/negative-vdot/exit-code.txt
1
PS C:\Users\33532>
```

（b）恢复正常参考模型：15 项检查通过

```
PS C:\Users\33532> wsl -d Ubuntu-22.04 -- tail -n 6 /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/functional-0/run.log
VDOT_SUMMARY mode=full contract=vdot-s8x8-r32-v0.1 planned=15 executed=15 passed=15 failed=0 unreachable=0
Core 0: HIT GOOD TRAP at pc = 0x8000236a
Core-0 instrCnt = 580586, cycleCnt = 737849, IPC = 0.786863
Seed=0 Guest cycle spent: 737853 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 1663260ms
PS C:\Users\33532> wsl -d Ubuntu-22.04 -- cat /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/functional-0/exit-code.txt
0
```

图 3-3　高位错误检出与正常参考模型恢复的原始日志

负控日志显示 `v10_high different`、ABORT 和退出码 1；差异 PC 为 `0x80001bdc`，ABORT 报告 PC 为 `0x800016b0`。其中 `right/wrong` 是比较器两侧标签，本例的扰动由参考端注入。正常日志显示 15 项全部通过、`HIT GOOD TRAP` 和退出码 0。原记录分别为 clean-02 的 `negative-vdot` 与 `functional-0`；截图及对应运行索引随源包保存。

3.4 全核波形中的点积事务

为观察点积在全核中的输入、响应和提交，2026 年 11 月 5 日使用稳定八项点积版本的 clean-02 封存生成 RTL，单独构建了支持 FST 的 EMU。1681 项生成输入与原清单一致；该模型运行原 functional-2 镜像，加载正常 NEMU 参考库并启用 DiffTest，最终到达 GOOD TRAP、退出码 0。以下波形将同一笔正数点积从执行输入关联到架构提交。

3.4.1 源数据、结果与身份

两个源寄存器均为 0xa5a5a5a5a5a5a5a50807060504030201，低八个有符号字节依次为 1 至 8。预期结果为

$$1^2 + 2^2 + \cdots + 8^2 = 204.$$

实际 128 位响应为 0x000000000000000000000000000000cc，符合低 32 位保存结果、高 96 位清零的布局。高 64 位 A5 哨兵同时检验了源元素选择范围。

关联字段	观测值
ROB 与物理目的	ROB0:39；物理目的 103（十六进制 67）
提交观察	`io_valid=1`、`io_vecwen=1`；架构目的 v10
PC 与机器字	PC=`0x80001b8c`；机器字 `0x0294050b`

3.4.2 从有效区间到握手边沿

波形中的 valid 表示一段有效区间，事务接受则发生在 valid 与 ready 同时有效的时钟上升沿。两者按下表区分，便于与日志中的周期坐标对照。

事件	显示坐标（ps）	观察依据
输入有效（A）	678694	输入 valid、ready 均为 1，源值及身份有效
输入接受边沿	678696	边沿前握手有效，边沿后 P 槽有效
输出有效（B）	678698	R 槽有效，输出 valid、ready 均为 1，结果 204
输出握手边沿	678700	边沿前输出有效，边沿后 R 槽清空
DiffTest 提交观察（C）	678710	提交有效，PC、编码、ROB 与目的相符

同一事务的 O2 日志记录接受、响应和内部 ROB 提交周期分别为 339296、339298、339301。接受到响应相隔两个全局模型周期，与第二阶段的两槽时序一致。内部 ROB 提交对应模型坐标 678706，DiffTest 观察口在 678710 有效，反映两拍观察流水延迟。分析执行时延时使用实际握手和内部提交坐标，分析 DiffTest 接口时使用其观察坐标。

图上方为全局时钟，下方为点积局部门控时钟；局部空闲时可停止时钟。显示单位 ps 是仿真模型的时间坐标，相邻全局周期相隔两个坐标单位。程序性能采用 DUT 周期，物理时序由综合和 STA 评估。

以下三幅 GTKWave 截图分别展示输入、响应和提交。截图保留信号值，仅裁去层级栏和外部空白；完整原图、事务采样、运行回执与文件摘要见 evidence/vdot-wave-20261105/和 figures/source-screenshots/。

3.4.3 输入与接受

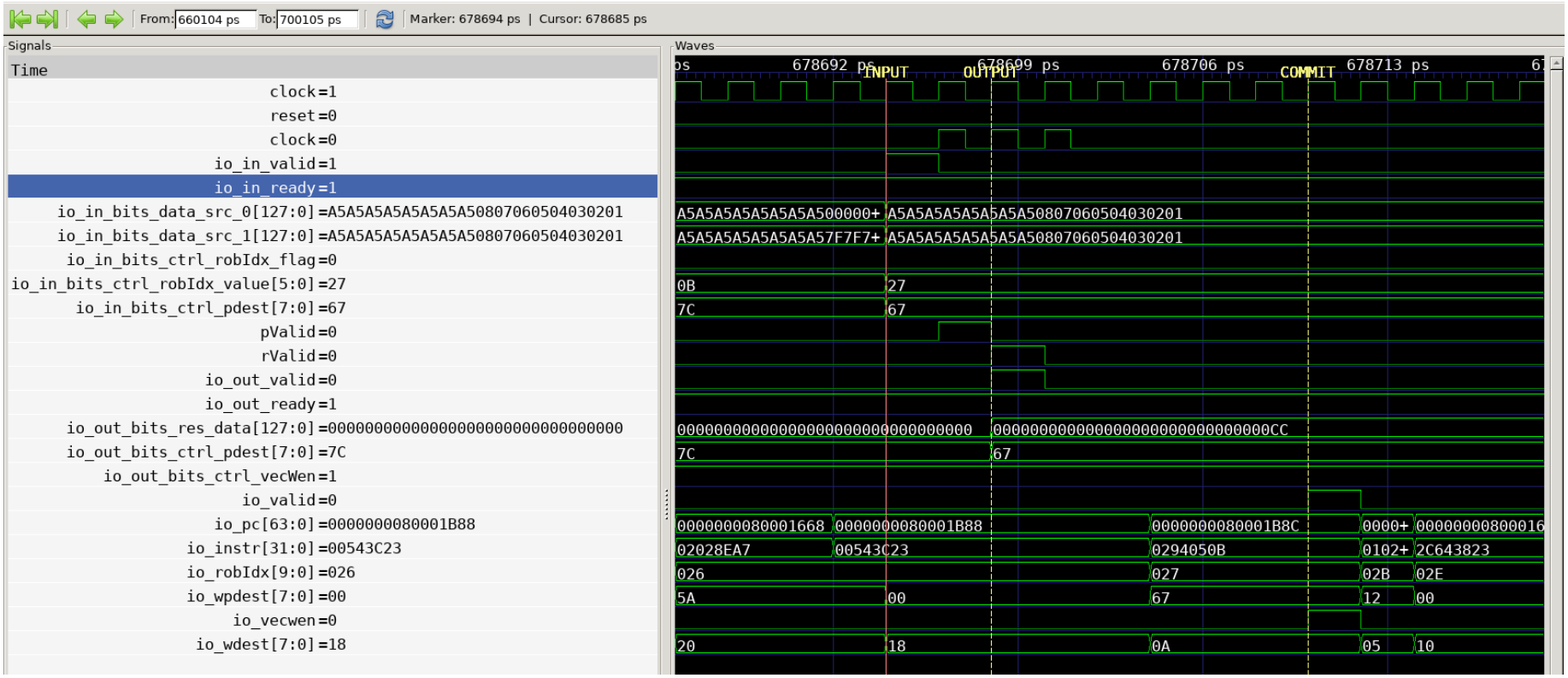


图 3-4　正数点积的输入与接受边沿

游标 678694 处输入 valid、ready 为 1；源低八字节为 1 至 8，ROB 值 27 和物理目的 67 按十六进制显示。678696 上升沿接收请求，P 槽有效。

3.4.4 结果与响应

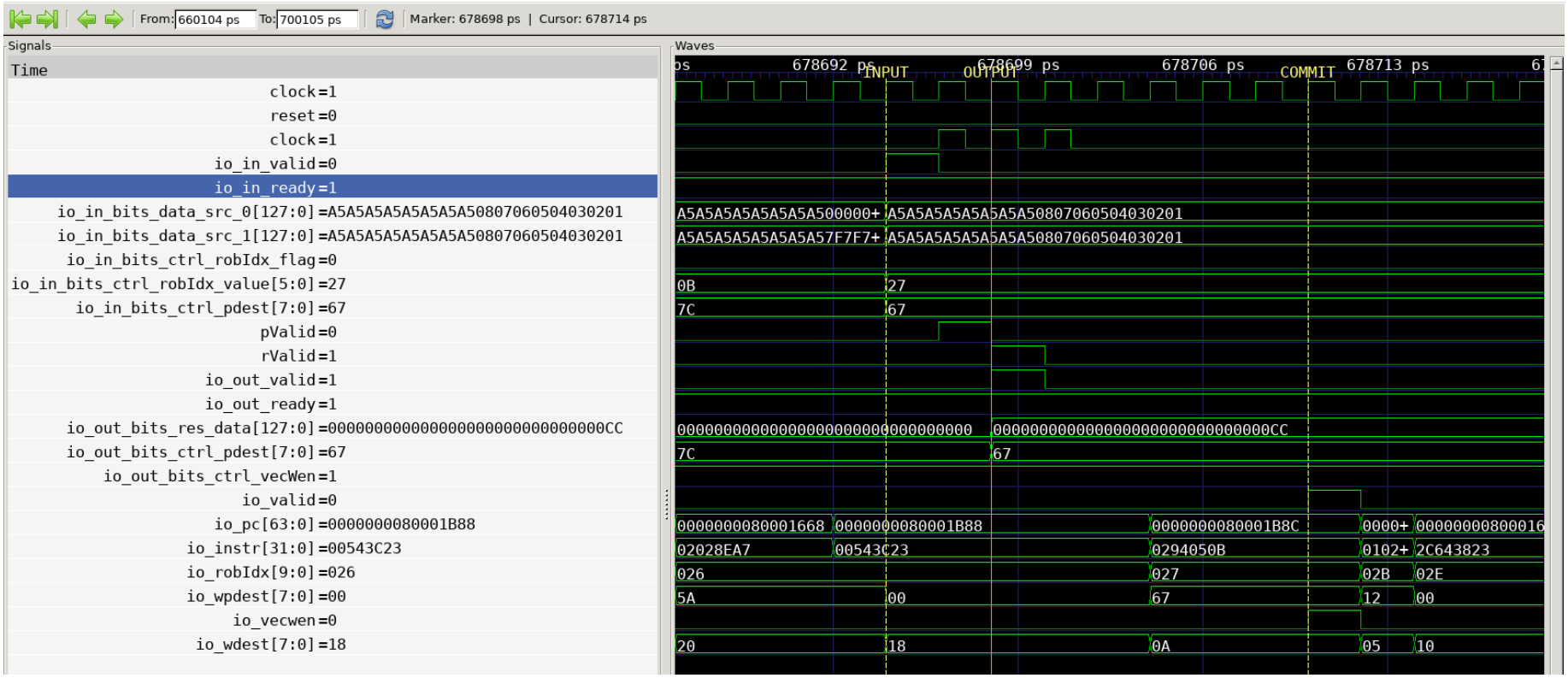


图 3-5　完整 128 位结果与响应握手

游标 678698 处 R 槽及输出 valid、ready 有效，结果为 204 且高 96 位为零；678700 边沿完成握手并释放 R 槽。此时提交观察口 io_valid 为 0。

3.4.5 提交与架构目的

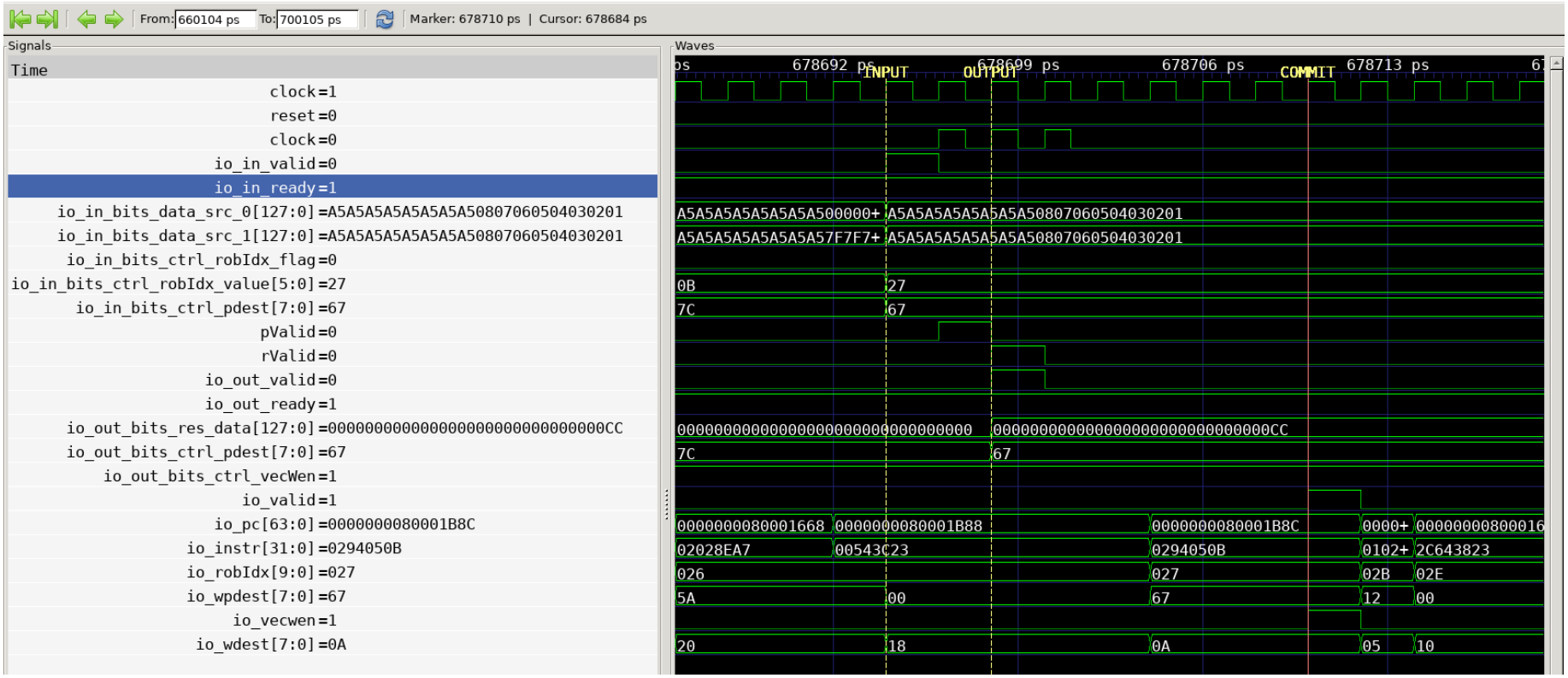


图 3-6 DiffTest 提交观察与架构目的

游标 678710 处提交有效，PC80001B8C、机器字 0294050B、ROB027、物理目的 67、架构目的 0A（v10）及向量写使能对应同一事务。观察口的两拍流水延迟已在正文对齐。

3.5 等价 RVV 对照与测量方法

8 项点积微基准中的标准 RVV 对照，通过宽乘和宽归约计算与 VDot 相同的有符号点积。单次已知配置序列先以 `vwmul.vv` 得到 16 位乘积，再切换到 `e16/m1/VL8`，清零整个目的寄存器并执行 `vwredsum.vs`，最后恢复入口配置。高 96 位在归约过程中保留为零，低 32 位保存完整整数和。

该微基准按使用场景组织：已知配置的单次点积为 5 条主体指令，通用入口增加 `vl/vtype` 保存，共 7 条；四个独立点积共享配置切换，主体合计 14 条。批处理中的配置复用使标准 RVV 也能充分利用独立计算，而比较仍覆盖双方实际执行的清零、搬运和状态管理。

3.5.1 完整窗口与配对统计

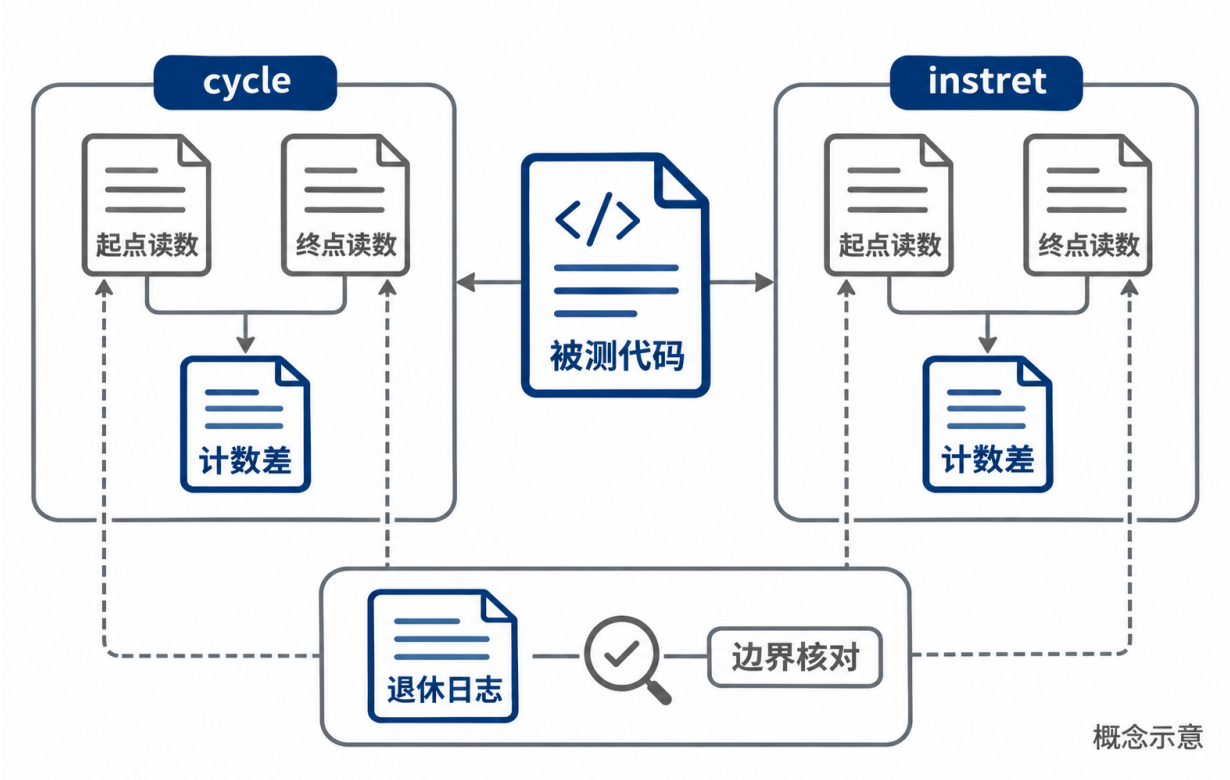
测量采用同一输入、循环次数、镜像配置和边界规则。每个微基准场景使用三组数据，每组统计四个正式窗口；首次、预热和独立重放分别保存。计时保留实际执行的配置切换、循环与边界开销，统计单位为 DUT 全局周期。

香山是乱序处理器，计数器的读取时刻需要与指令执行位置对应。项目在固定受测实现上组合使用 fence 与 cycle、instret 读数，并通过观察器关联四条 CSR 指令的 PC、机器字、ROB 身份、采样和退休事件。cycle 差与全核采样 tick 差相符，instret 差与退休记录及 ELF 展开计数相符；绝对计数器使用整数解析。该方法的适用对象由受测硬件和测量合同共同确定。

加速比与周期降低率分别定义为

$$S = \frac{C_{\mathrm{RVV}}}{C_{\mathrm{VDot}}}, \qquad R = \frac{C_{\mathrm{old}} - C_{\mathrm{new}}}{C_{\mathrm{old}}}.$$

二者使用不同分母，表中同时给出原始周期与比较对象。完整窗口还包含调度、访存、循环和边界开销，因此与第二阶段的 FU 两拍最早响应分别报告。



概念示意：测量窗口与执行边界。技术依据见图 3-7。

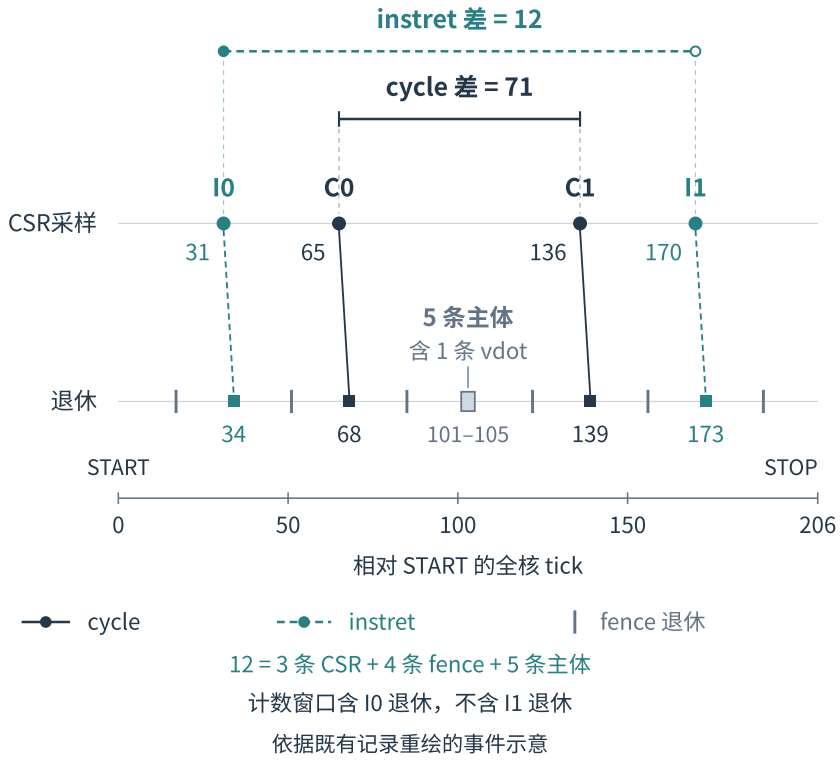


图 3-7　CSR 采样、退休与计时窗口的对应关系

依据观察器事件重绘。所示窗口的 cycle 差为 71、instret 差为 12；图中分别标出进入 ROB、CSR 采样与退休事件。

3.6 硬件微基准结果

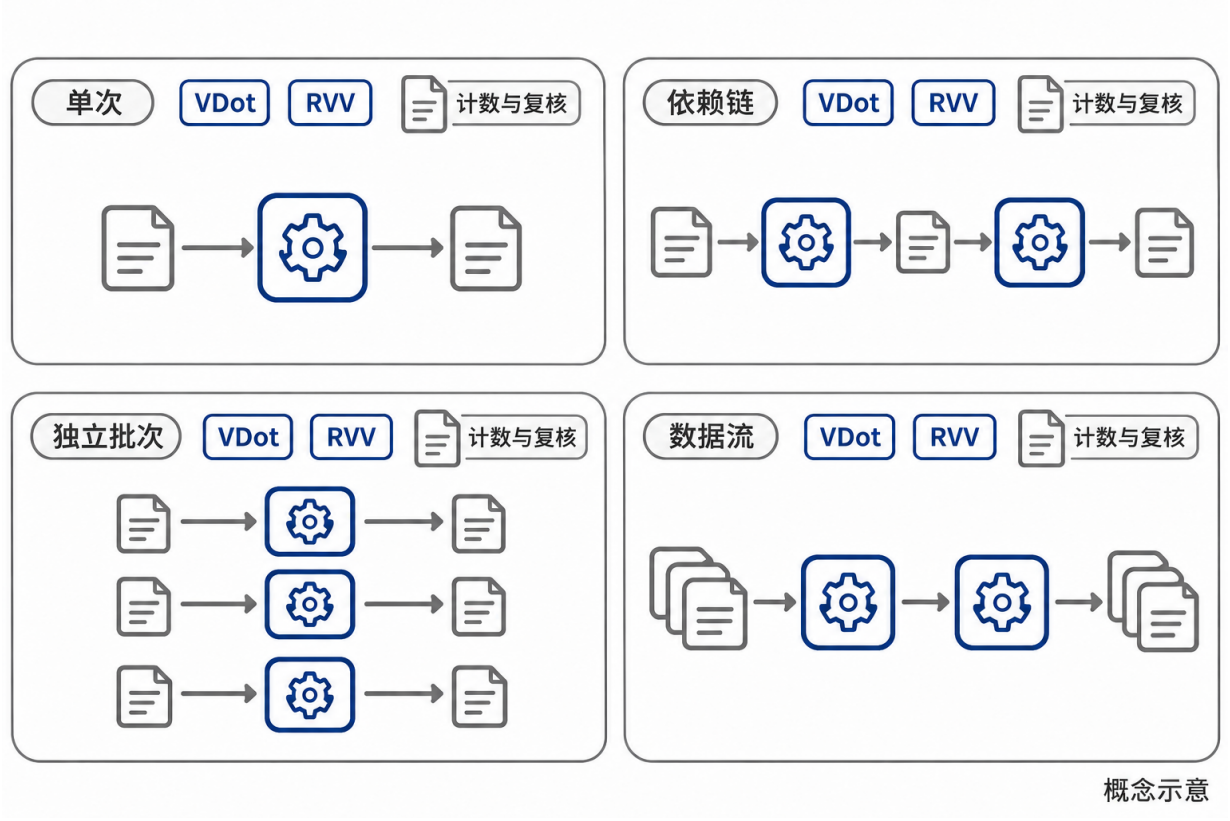
四类微基准分别观察单次点积、连续数据依赖、独立任务吞吐，以及加载、计算与存储共同作用下的表现。表 3-3 使用同镜像中的 VDot 与等价 RVV 路径；每个场景、每种实现使用三个数据集，每个数据集取四个正式窗口，共十二窗。

表 3-3 0f 受测实现的微基准结果（周期均值）

场景	VDot	RVV	加速比
单次点积	71.00	85.25	1.2007×
依赖链（256 次）	1,857.00	5,440.50	2.9297×
四路独立批量（128 次）	221.00	678.00	3.0679×
加载、计算、存储流（64 次）	1,127.50	1,133.00	1.0049×

加速比按同一场景的 RVV 周期总和除以 VDot 周期总和计算。独立批量由 32 轮四路点积组成，能够持续向流水线提供相互独立的工作；依赖链则要求每次点积使用前次结果。两者分别呈现了吞吐率和依赖关系对收益的影响。加入加载、存储与循环控制后，stream64 两条路径的周期接近，说明仅缩短整数点积还不足以决定整个计算循环的速度。

在单独的 O1 与 O2 同镜像比较中，独立批量由 362.5 周期降至 221 周期，减少 39.03%。在 0f 与其修复前版本的独立复核中，对同一实现、同一输入、同一序号的 96 对正式窗口进行比较，周期差值均为零。这些结果提示，应用优化还需同时考察任务独立性、数据搬运和循环开销。



概念示意：统一条件下的性能比较。技术依据见图 3-8。

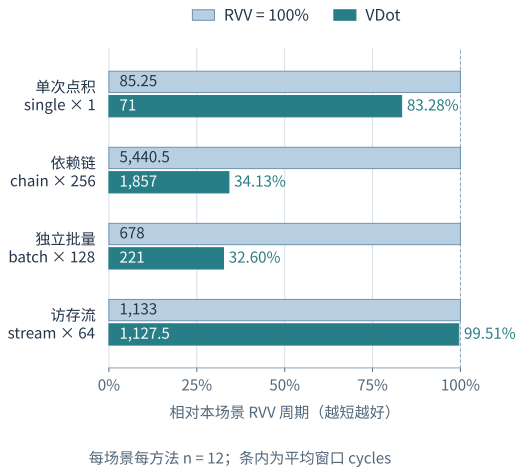


图 3-8 四类微基准周期与 RVV 归一化结果

每个场景以 RVV 周期为 100%；条内列正式窗口的平均 DUT 周期，条尾列 VDot/RVV 周期百分比。加速比见本节表格。

3.7 软件调用与 C8 优化

前述微基准考察 8 项点积的指令序列，本节转向 8 行、每行 32 项 GEMV 的完整函数调用。硬件点积单元要在矩阵运算中发挥作用，还需要减少相邻计算之间的配置和数据搬运。firstdot 用首段点积直接初始化累加状态，随后接续其余分段；C8 在复用输入向量的 8 行、每行 32 项 GEMV 路径中，进一步消除重复的向量配置。两者分别减少累加初始化与配置管理的开销。

C8 提供普通 RV64 C 接口。调用者将行优先矩阵 $A[8][32]$ 和输入向量 $x[32]$ 连续存放，形成 288 字节输入；返回值写入 8 个 16 字节结果槽，每槽低 32 位保存 INT32 点积，高 96 位清零。固定槽布局与指令的完整结果格式一致，便于在功能检查和后续处理之间传递结果。

VDot 路径将每行分为四个 8 项点积段；RVV 路径则处理两个 16 项半组，以 `e8/m1/VL16` 宽乘和 `e16/m2/VL16` 宽归约接续得到完整 32 项组和。两种实现共享输入布局与完整结果格式，各自采用适合的分组和配置复用方式。

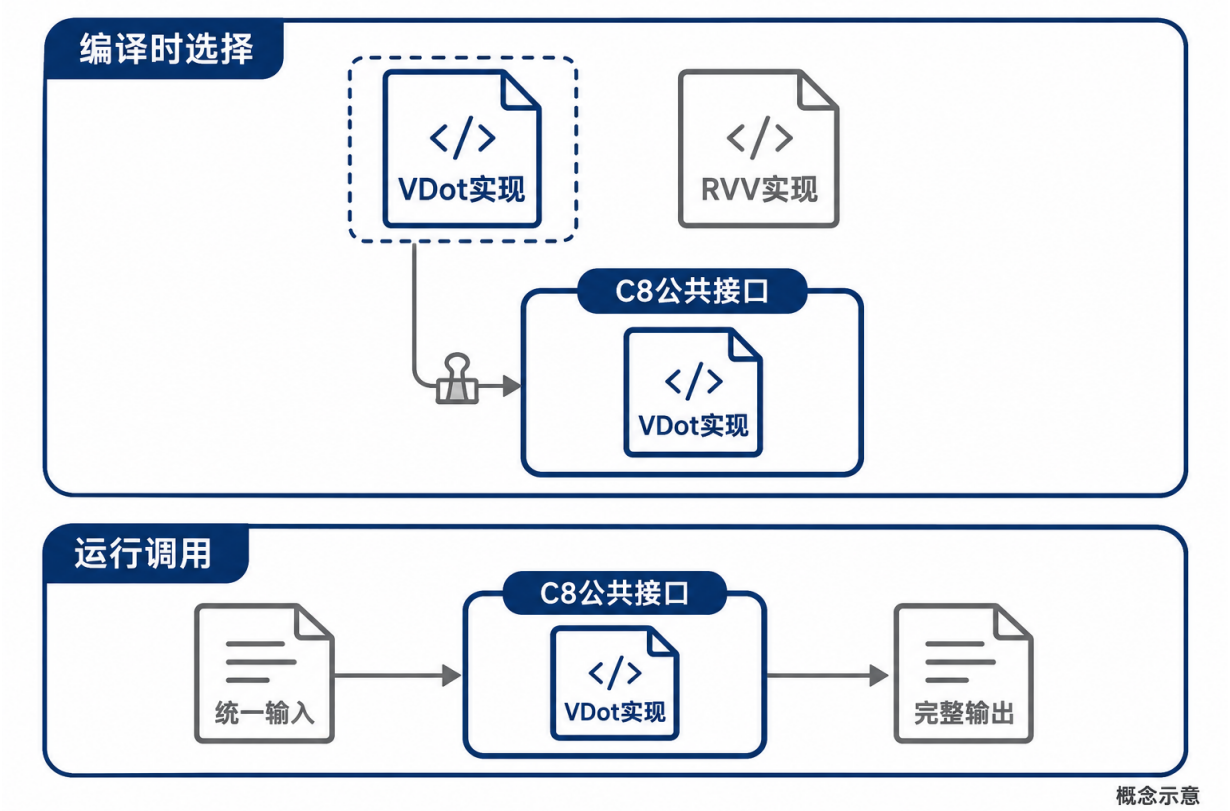
8×32 整数 GEMV 调用	周期均值	对照意义
firstdot	282.25	首段直接初始化
C8 配置复用 VDot	273.00	本节选用实现
同批较快合格 RVV	299.00	标准指令对照

在稳定八项两槽实现的这组 8×32 配对测量中，输入已打包为 INT8，输出为 8 个完整结果槽。C8 相对 firstdot 减少 3.2772% 的周期，相对较快 RVV 的加速比为 1.095238。统计采用三个数据集、每组四个正式窗口，包含完整调用开销。这些结果说明，整数点积之外的软件组织仍然影响最终收益。[1]

接入时，`xs_c8_gemv_8x32` 在编译期选择对应函数，显式 RVV 入口提供同接口对照。原合格程序、源码复制对照和 SDK 接入版的 31,440 字节 BIN 逐字节一致，装载段、48 项调用表和

[1]C8 接口和结果见C8 软件包验收结果及其固定输入清单。这里的周期对应该 8×32 调用计划。

12 组输入也一致。[2]这条从函数对象到最终镜像的核查链，使接口封装与实际执行代码建立了明确对应关系。新应用可据此复用固定函数，并针对自己的准备流程和调用窗口开展整体验证。



概念示意：公共接口与缓冲区组织。技术依据见图 3-9。

[2]三方最终镜像比较见C8 最终镜像等价结果。

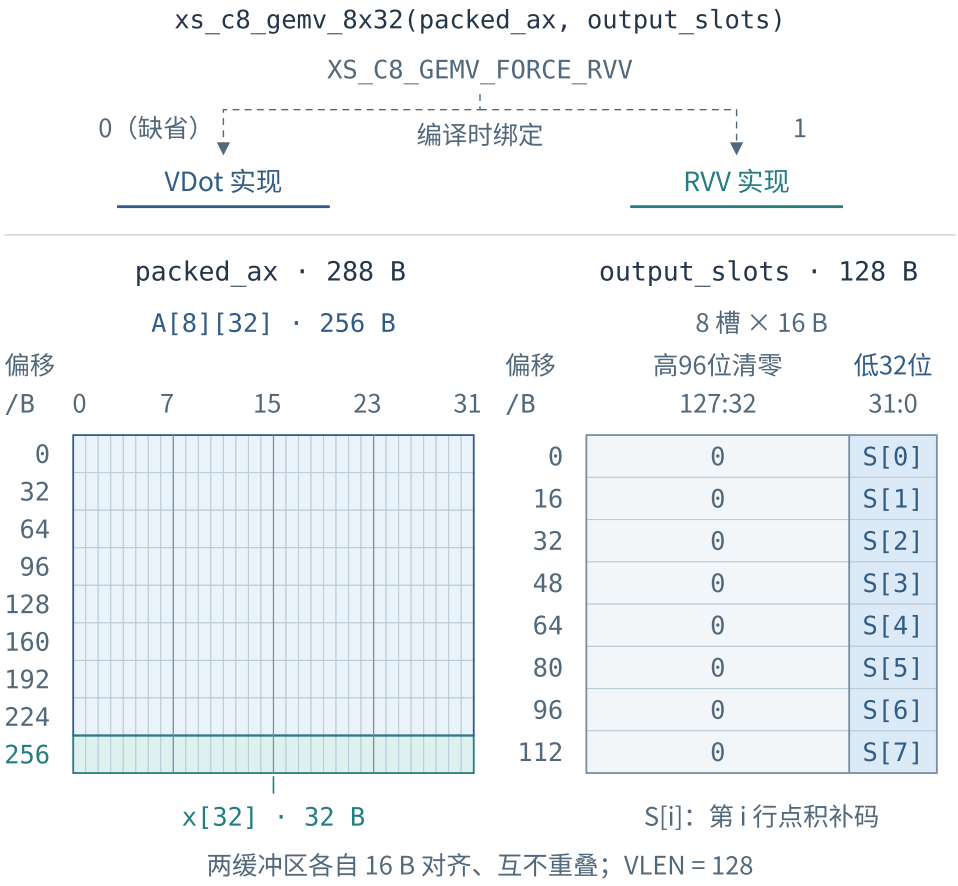


图 3-9 firstdot 与 C8 的软件入口和结果布局

C8 读取连续的 INT8 矩阵与向量，输出 8 个完整 128 位结果槽。编译期入口分别连接 VDot 或 RVV 函数，调用者使用同一缓冲区布局。

3.8 软件接入与复现说明

3.8.1 固定平台与数据布局

C8 软件包面向本项目已验证的 RV64、lp64d、VLEN128 裸机环境，调用前启用向量状态。VDot 路径使用项目的自定义点积编码；RVV 路径使用标准向量指令。输入、输出各按 16 字节对齐并独立存放，输入在调用期间保持有效。

项目	接口约定
输入	行优先 $A[8][32]$ 后接 $x[32]$，共 288 字节
输出	8 个 16 字节槽，共 128 字节
单槽	低 32 位 INT32 结果，高 96 位零
返回向量配置	`vl=8, vtype=0, vstart=0, vcsr=7`

函数沿用普通 RV64 C ABI 保存栈和被调用者保存寄存器。向量寄存器按该接口供计算使用，返回配置固定为 `e8/m1/tu/mu`。结果槽宽度对应 VLEN128 下的整寄存器存储，因此调用方按照 16 字节步长读取各行结果。

3.8.2 编译期选择与调用

头文件 selected_gemv_c8.h 提供两个入口。xs_c8_gemv_8x32 默认连接配置复用 VDot；将 XS_C8_GEMV_FORCE_RVV 设为字面量 1 后，该名称连接 RVV 实现。xs_c8_gemv_8x32_rvv 始终连接 RVV。开关为 0 或 1，选择过程在预处理阶段完成。切换版本或开关后，重新编译相关翻译单元并链接匹配的静态库。

```
#include <stdint.h>
#include "selected_gemv_c8.h"

struct c8_slot {
    int32_t value;
    uint32_t upper_zero[3];
};
static _Alignas(16) int8_t packed_ax[288];
static _Alignas(16) struct c8_slot result[8];

/* Fill packed_ax with A[8][32] followed by x[32]. */
void run_c8(void) {
    xs_c8_gemv_8x32(packed_ax, result);
}
```

需要标准 RVV 对照时，可在同一调用位置使用显式 RVV 入口。保留 firstdot 版本的应用继续使用其对应头文件和库；C8 以独立名称加入软件包，便于按具体形状选择和回退。[3]

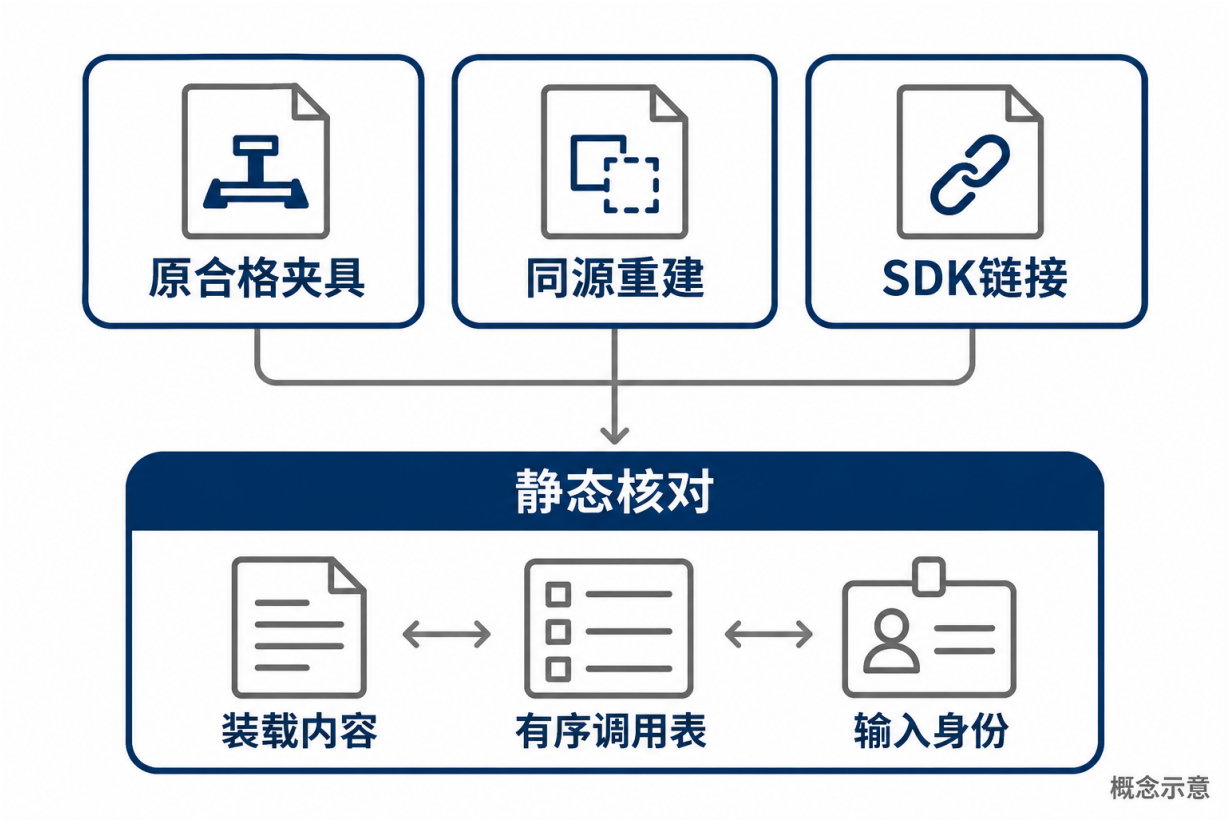
3.8.3 从函数对象到可执行镜像

软件包保留一个完整 gemv.o，由 libxs_gemv_c8.a 提供 16 个公共函数。固定 GNU 参数为-march=rv64gcv -mabi=lp64d -mno-relax。验证分别核对编译期别名、实际调用重定位、库成员抽取和最终装载内容，使每个公开入口都能追溯到被测试的函数。

SDK 接入的三方比较覆盖 BIN、PT_LOAD、BSS、函数地址与机器码、调用表及输入数据。三份 BIN 均为 31,440 字节且内容一致；去除调试节后的 ELF 也逐字节一致。源目录和头文件路径造成的调试信息差异单独保留。这使队友既能使用调试材料，又能核实接入前后的运行内容。[4]

[3]头文件及对齐示例见C8 编译期入口与同目录 example.c。接口细节见C8 合同。

[4]完整比较记录、链接映射及独立 GNU 复核见C8 最终镜像等价结果。



概念示意：三种制品分别核对装载内容、有序调用表和输入身份。技术依据见图 3-10。

图 3-10　C8 接口、静态库与最终镜像的核对流程

从头文件选择进入实际库成员，再比较最终装载段、调用记录及输入数据。三方对照连接了公开接口与已验证程序的运行内容。

复现时使用软件包的 README、固定输入清单及其配套核验工具，在独立输出目录保留构建结果。新的应用接入把模型准备、数据打包和计时窗口一并列入验证计划，按同一数据生命周期比较 VDot 与较快的 RVV 实现。

3.9　完整前向计算的组织与验证

3.9.1　从点积内核到模型计算链

应用验证将量化矩阵向量乘接入一个六层 Transformer 的完整 forward。固定合成模型的模型维度为 288，前馈中间维度为 768，注意力头数为 6，词表大小为 32,000；测试在规定初始化后，执行 token=1、position=0 的一次前向计算。权重与尺度预先完成打包，该准备成本不计入本次窗口。计数窗口包围一次完整 forward 调用，包含激活量化；计算依次经过嵌入读取、各层归一化、Q/K/V 投影、位置编码、注意力、输出投影及前馈网络，最终生成 32,000 项分类器输出。

本节的量化 GEMV 内核按每 32 项一组计算，直接生成 FP32 行输出。每组先求精确的 INT32 点积，再转换为 FP32，依次乘以权重尺度和激活尺度，最后按组号顺序累加到输出。不同内核共用权重、尺度数组及这一浮点计算顺序。Q、K、V 投影复用同一份量化激活，前馈网络的两个输入投影也共享量化结果。

3.9.2 工作量与数据布局

表 3-4 列出一次 forward 中的 43 个量化 GEMV。这里以一项乘加记一个 MAC，统计矩阵的有效维度；注意力、归一化、激活函数等运算另由完整 forward 承担。

表 3-4 固定模型一次 forward 的量化 GEMV 工作量

矩阵类别	数量	每个矩阵的形状	逻辑 MAC	占比
Q/K/V/O 投影	24	288×288	1,990,656	13.11%
前馈 W1/W3	12	768×288	2,654,208	17.48%
前馈 W2	6	288×768	1,327,104	8.74%
分类器	1	$32,000 \times 288$	9,216,000	60.68%
合计	43	—	15,187,968	100.00%

分类器占逻辑 MAC 的 60.68%，因此大矩阵路径是有价值的优化对象。这个比例描述乘加工作量；其实际耗时还取决于访存、尺度处理、循环控制及处理器执行资源。

现有布局将权重按八行和量化组打包，尺度沿每个输出行连续存放。输入维度为 288 的矩阵由 288 行面板处理，共调用 172 个面板；输入维度为 768 的六个 W2 矩阵使用对应内核。末面板保持相同计算形状，再复制有效输出。这种统一组织多计算 1,408 个填充行，使实际整数乘加项为 15,593,472，比有效工作量多 2.67%。

3.9.3 完整计算链的验证结果

固定模型已分别通过标准 RVV 路径 FR 和自定义 VDot 路径 TA 的目标机完整 forward 验证。两路均完成 3,691,424 字节的全状态比较，并核对输出保护区、调用状态及浮点异常标志。表 3-5 记录该正确性实验中各一次预设初始状态下的计数窗口；TA 独立重放得到相同的完整验证结果和窗口计数。

表 3-5 完整 forward 正确性实验中的单窗口目标机观测

实现	周期数	退休指令数
FR：标准 RVV	20,258,039	7,502,872
TA：自定义 VDot	19,774,245	10,122,100

这组观测将量化 GEMV 与其余模型运算放入同一条计算链，也显示退休指令数和周期数应分别分析。这些单窗口记录用于完整执行与复现检查，稳定的应用性能比较还需进一步测量。

在相同模型与共同程序上，进一步接入了两种标准 RVV 软件组织：GS32 一次处理完整的 32 项量化组；paired16 把前后半组对应的乘积在 16 个 INT16 通道中相加，再扩宽归约。paired16 在入口检查激活值，遇到 -128 时执行完整 FR 路径，保持完整的 signed-int8 数值域。这些入口检测和回退属于实际调用的一部分。

新生成的三路共同程序已完成 NEMU 参考验证，三个正常方法的完整输出与状态逐位一致。表 3-6 给出该组参考执行的窗口退休指令数；其中 W2 保持 FR，实现差异集中在输入维度为 288

的矩阵路径。参考指令数用于评估软件组织；该组香山目标机周期结果尚待完成验证。

表 3-6 共同参考模型中的完整 forward 指令数

实现	窗口退休指令数	相对本组 FR 的指令数减少
FR	7,502,886	—
GS32	4,828,788	35.64%
paired16	5,175,371	31.02%

共同验证还包含 paired16 重放与末项输出扰动负控，分别检查结果复现和比较器的拒错能力。

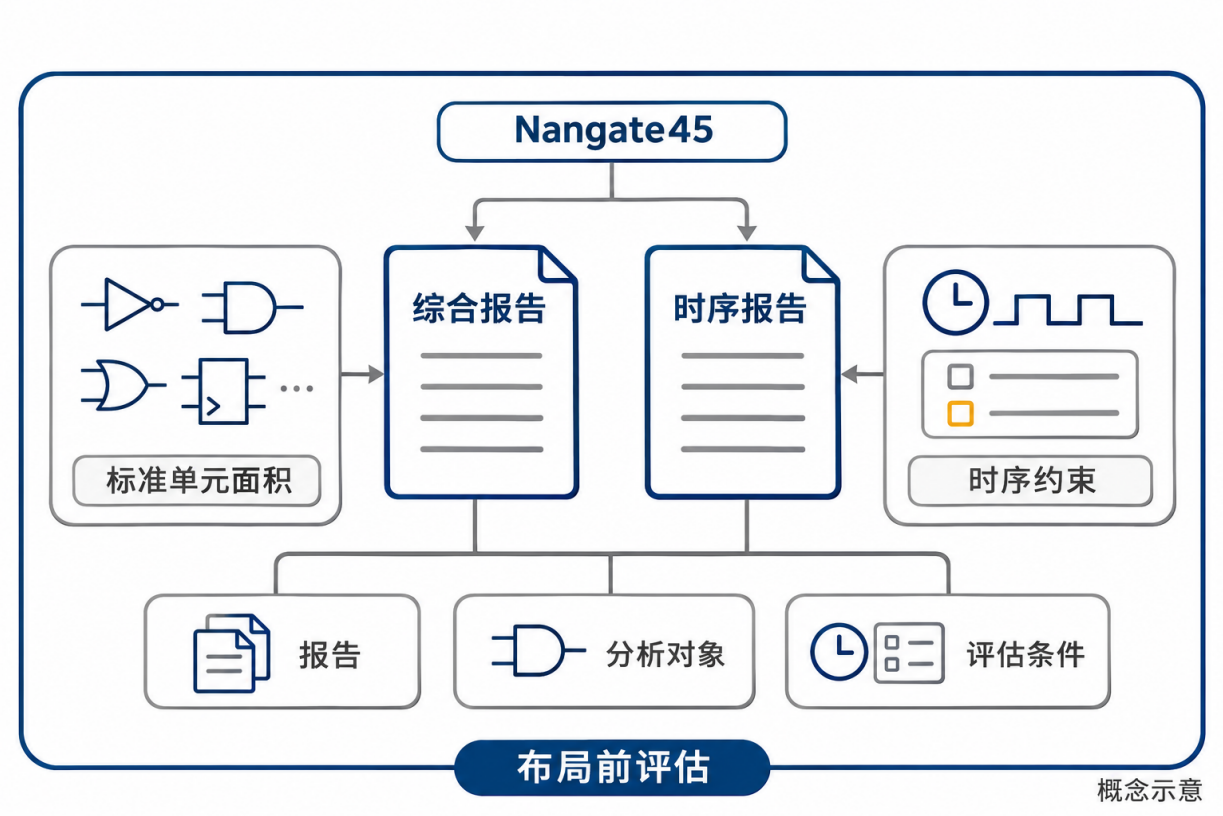
3.10 综合面积、时序与设计成本

设计成本使用 Nangate45 Typical 库，条件为 1.10V、25 °C，工具为 Yosys 0.9、ABC 1.01 和 OpenSTA 2.0.17。受测对象是带 32 位结果保存和 DIV 取消修复的八项点积两槽版本。主档采用 1 ns 时钟周期、5 fF 负载及固定线负载模型，另以 0.5 ns、2 ns 周期和 1 fF、20 fF 负载检查敏感性。

指标	独立 VDot FU	共享 VFEX4 父级
标准单元面积（μm^2）	17298.246	284614.946
寄存器至寄存器到达时间（ns）	1.163404	3.261218
该类路径 setup 裕量（ns）	−0.240117	−2.341112
所报告路径最差 setup 裕量（i2r，ns）	−0.268520	−2.393998

两个统计范围具有包含关系：VFEX4 覆盖 VDot、DIV/SQRT 及共享控制。最差 setup 裕量来自输入至寄存器路径，与表中的寄存器至寄存器路径分别列示。1 ns 主档下两种统计范围均存在 setup 违例，后续物理实现需要继续进行时序收敛；本组映射和 STA 数据用于比较相同工艺与约束下的结构成本。

将结果状态从 128 位收窄为 32 位后，独立 FU 面积相对其前一版减少 2.33386%，共享父级减少 0.137349%。进一步加入 DIV 取消修复时，独立 FU 面积保持 17298.246 μm^2，父级由 284879.882 降至 284614.946 μm^2；对应 96 对正式微基准窗口的周期逐对相同。控制正确性、面积和各类时序路径共同支持设计取舍。



概念示意：面积与时序的不同约束。技术依据见图 3-11。

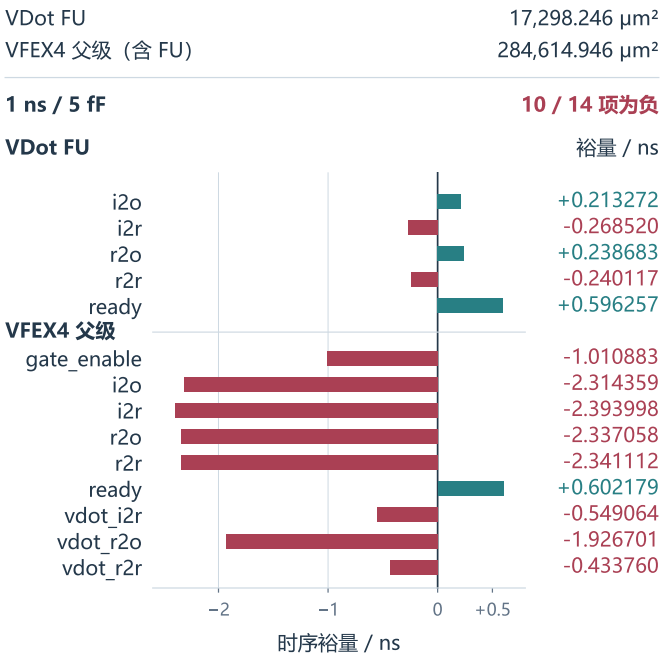


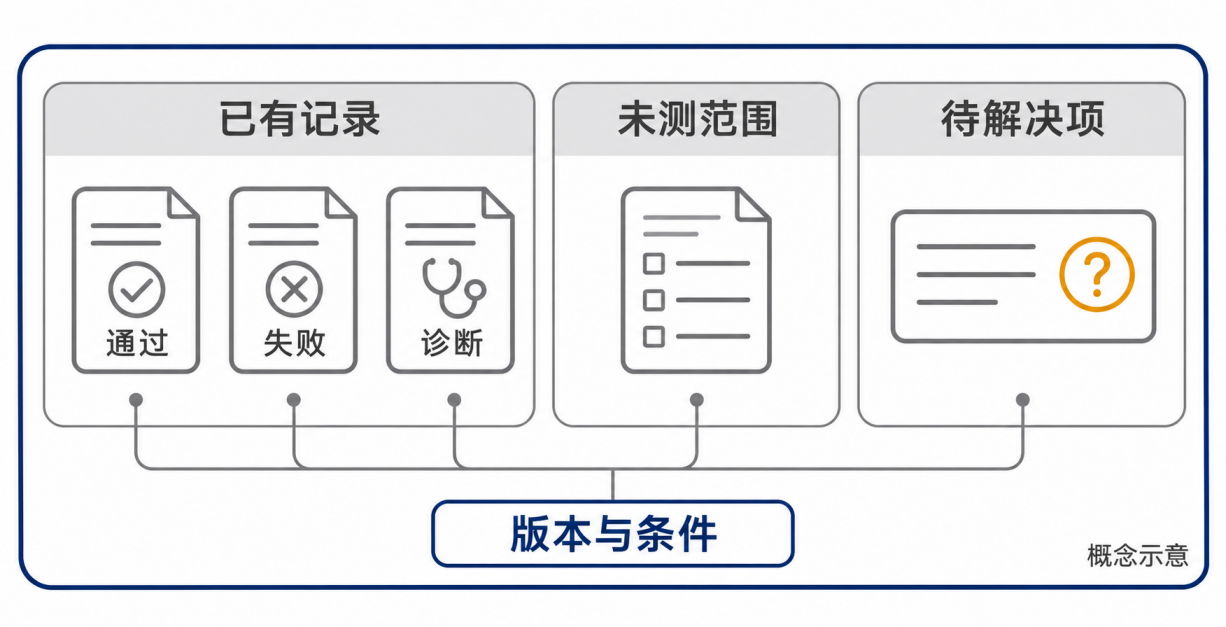
图 3-11 Nangate45 模块面积与主档 STA 裕量

主档采用 1ns 周期、5fF 负载。图中列出 FU 和包含 FU 的完整 VFEX4 父级面积，以及 14 类路径的 setup 裕量，其中 10 类为负值。

3.11 设计取舍与应用方向

两槽流水改善独立点积的持续供给，32 位结果保存利用了指令布局中的常零高位，DIV 取消修复完善共享资源的释放。软件侧则通过 firstdot 初始化、C8 配置复用及更适合量化组的 RVV 组织减少实际调用开销。各项优化作用在不同环节，配对实验使其收益与代价能够分别解释。

固定 GEMV 已经给出普通 C 调用的周期结果，完整前向计算进一步验证了量化内核与模型计算链的衔接。最新十六项点积候选完成了指令和执行通路的功能验证，后续应用比较将继续保留较快的标准 RVV 对照。布局后时序、整核频率和功耗则属于进一步物理实现的评价内容。



概念示意：已知证据与尚未覆盖的边界。技术依据见图 3-12。

八项点积受测组合 0f + firstdot V2 + C8

● 已有记录　○ 待评估　— 随所属硬件评估

证据层级	逻辑与执行		物理实现		
	功能正确性	周期观察	单元面积	布局前 STA	功耗
局部 VDot FU 算术与握手	●	●	●	●	○
VFEX4 父级 含 FU 与 DIV	●	●	●	●	○
整核程序 差分／微基准	●	●	○	○	○
固定 GEMV / C8 完整调用	●	●	—	—	—

周期单位	FU 握手拍；整核与 C8 为 DUT 周期
功能记录	全核 117 用例／124 次；C8 最终镜像等价
时序结果	1 ns／5 fF：14 类中 10 类 setup 裕量为负
后续评估	整核 Fmax、布线后时序与功耗
复现方式	固定源码、镜像、输入及配套原始记录

图 3-12　八项点积受测组合的验证与评估范围

实心点表示该对象已有对应记录，包含未收敛的 STA 结果；空心点标出后续评估项目。FU 的局部握手拍与整核、C8 的 DUT 周期分别计量。

(a) FU/parent and full-core scope: evidence column of R5 matrix

0f真实FU与VFEX4父级；完整128位、反压/取消/身份/异常、严格DIV关钟及4+16负控；复用0f原167载荷；未新执行RTL仿真，不转授C1/C2/C3
固定原13项重新解析+新o2-control-diagnostic；117不同/124次功能例，O2覆盖及负控恢复；组合验收不是fresh14；原14项UTC容量FAIL保留；本次UTC间隔比单调时钟短62.888356秒，系统墙钟稳定性未证实，新占用审计采用单调包络

(b) Cost scope: evidence column of R5 matrix

0f同约束面积/五档STA资料1008文件回读；FU17298.246、父级284614.946微米平方；不是新综合；1ns负slack保留，整核Fmax/布线后时序/功耗NOT_TESTED

图 3-13　八项点积受测组合的验收与成本记录摘录

上部列出 FU、父级及整核功能记录；下部列出标准单元面积和布局前 STA 的评估范围。对应结果在本章按测试对象和指标分别说明。

3.12　评估对象与证据索引

报告按实际受测对象组织结果。四类微基准、普通 C 调用和完整 forward 各自对应固定的代码、程序与输入；可沿下列索引定位产生每组结果的实验。

评估对象	固定身份	对应结果
0f 硬件微基准	硬件提交 `0f12df6753` 及该批冻结 EMU	四类微基准的正式窗口和同镜像比较
R5 工程复现	`clean-02` 构建及其参考模型	干净环境构建、差分和工程验收
C8 普通 C 入口	最终 BIN 的 SHA256 前缀 `56805506eb98`	固定 GEMV、调用状态与最终镜像等价
FR/TA 完整 forward	固定 T17 环境与 FR/TA 消费者程序	首个 token 完整执行、全状态比较及目标机单窗口观测
三路 RVV 完整 forward	共同 FR/GS32/paired16 程序及固定模型	全状态比较、重放、负控及参考指令数

四类微基准可由微基准原始窗口 CSV与测量方法与成本汇总复查；软件包的使用与版本选择见C 入口与版本间比较。

完整前向计算的FR/TA 目标机记录和FR/GS32/paired16 共同参考结果分别保存目标周期与参考指令计数。两个摘要也随 LaTeX 整合包提供；原始仓库路径与 SHA256 见同目录的 `provenance.json`。各实验目录同时保存构建身份、输入清单和原始运行记录。

3.12.1 复现与图像来源

本报告在 `zjhwork` 文档与证据基础上，合入 `zufan` 的功能进展和已选插图，资料修订日期为 2026 年 11 月 6 日。实际产生测量的构建由前表单独标识，仓库合并与文档编辑不改变这些实验的身份。

原始日志、失败记录及构建清单随各实验目录保存。R5 组合验收保留原 UTC 占用审计记录，并使用同一启动环境下的单调时钟记录补充核对；性能表使用 DUT 周期。复现者可从各目录的 RESULT、合同和输入清单进入对应原始记录。

图中概念示意用于解释计算和验证流程，技术图依据源码、测试记录及测量数据绘制，终端与 GTKWave 截图保留实际观察内容。LaTeX 整合包包含正文直接引用的图像与日志，完整工程证据按本节索引在团队仓库及对应归档中查询。